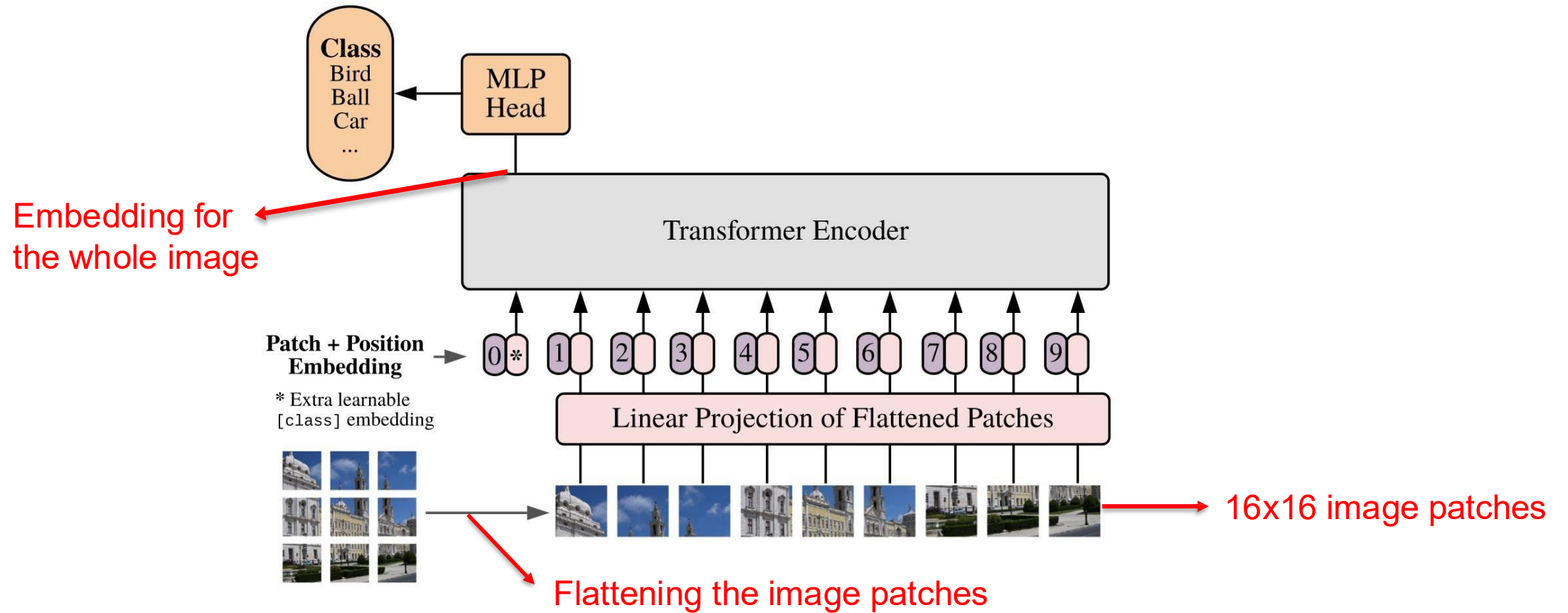


Deep Learning

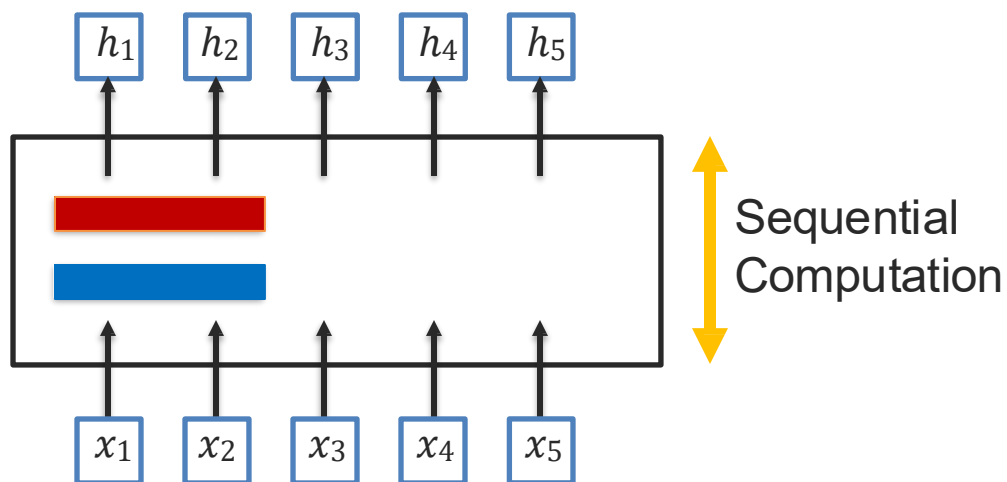
Lecture 9

Vision Transformer



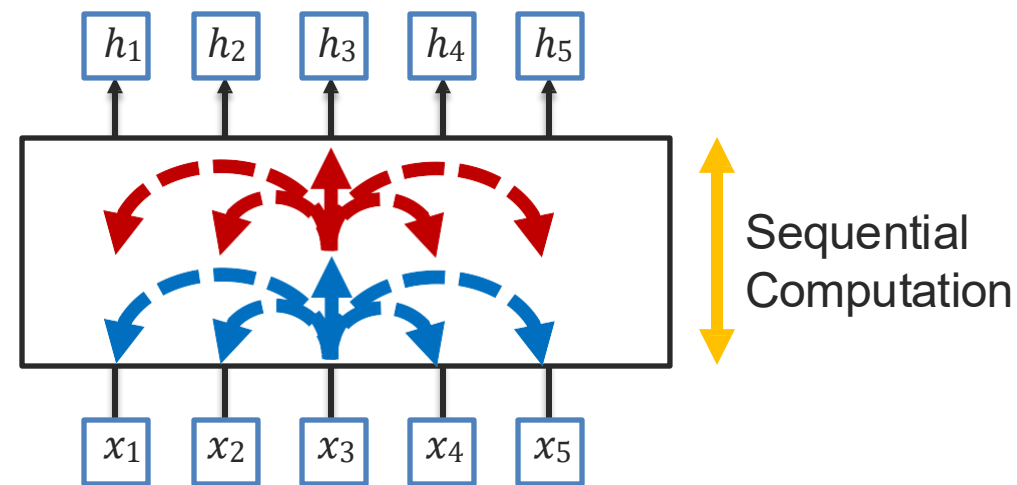
CNNs vs Transformers

Convolutions



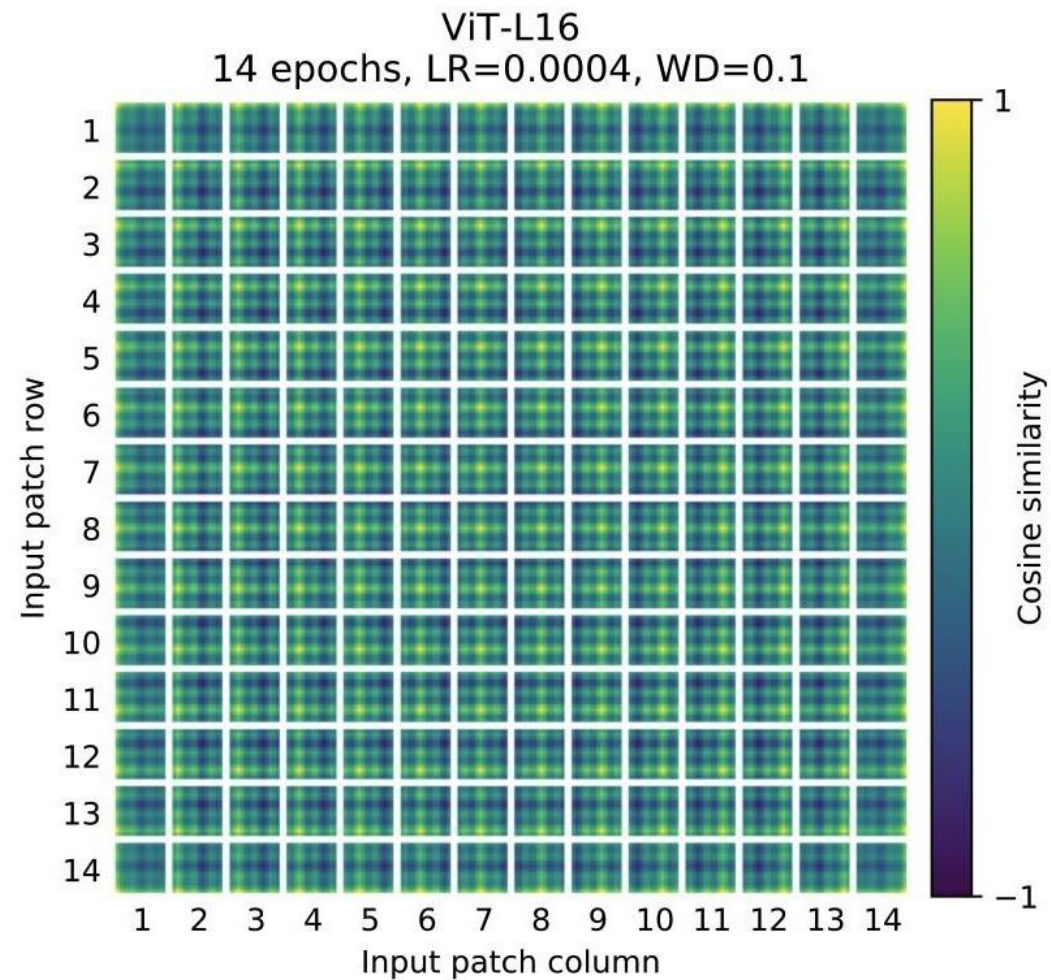
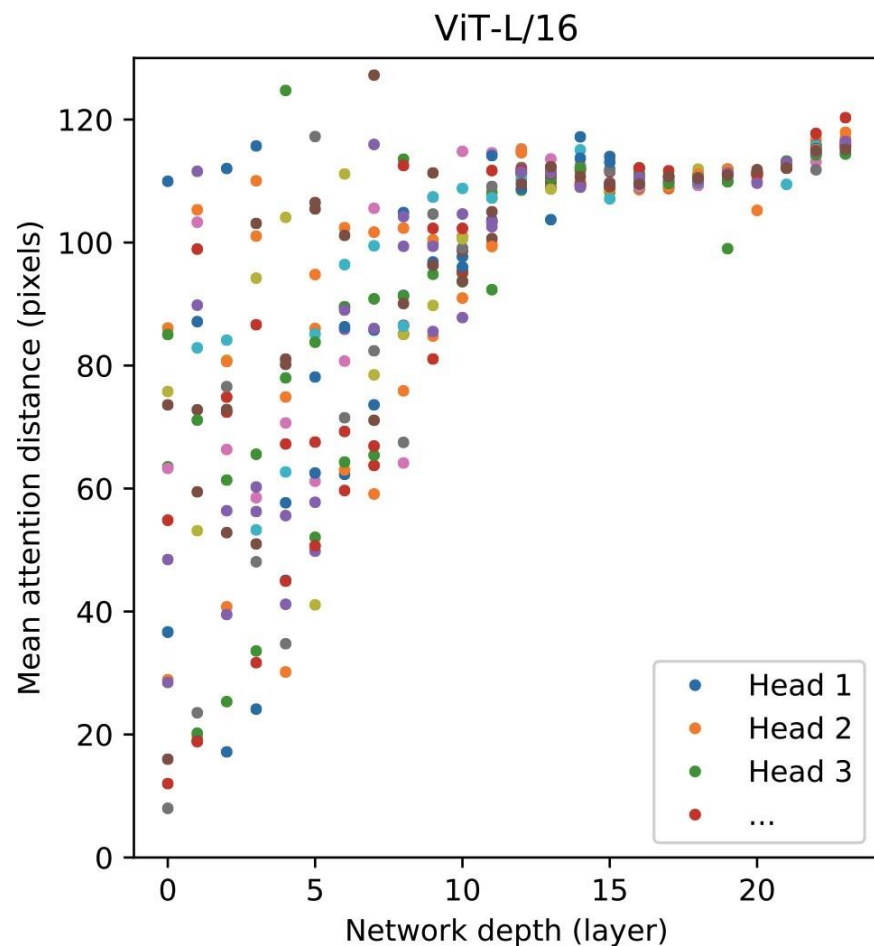
Modeling long-range dependencies requires many layers.

Self-attention



No inductive bias toward locality

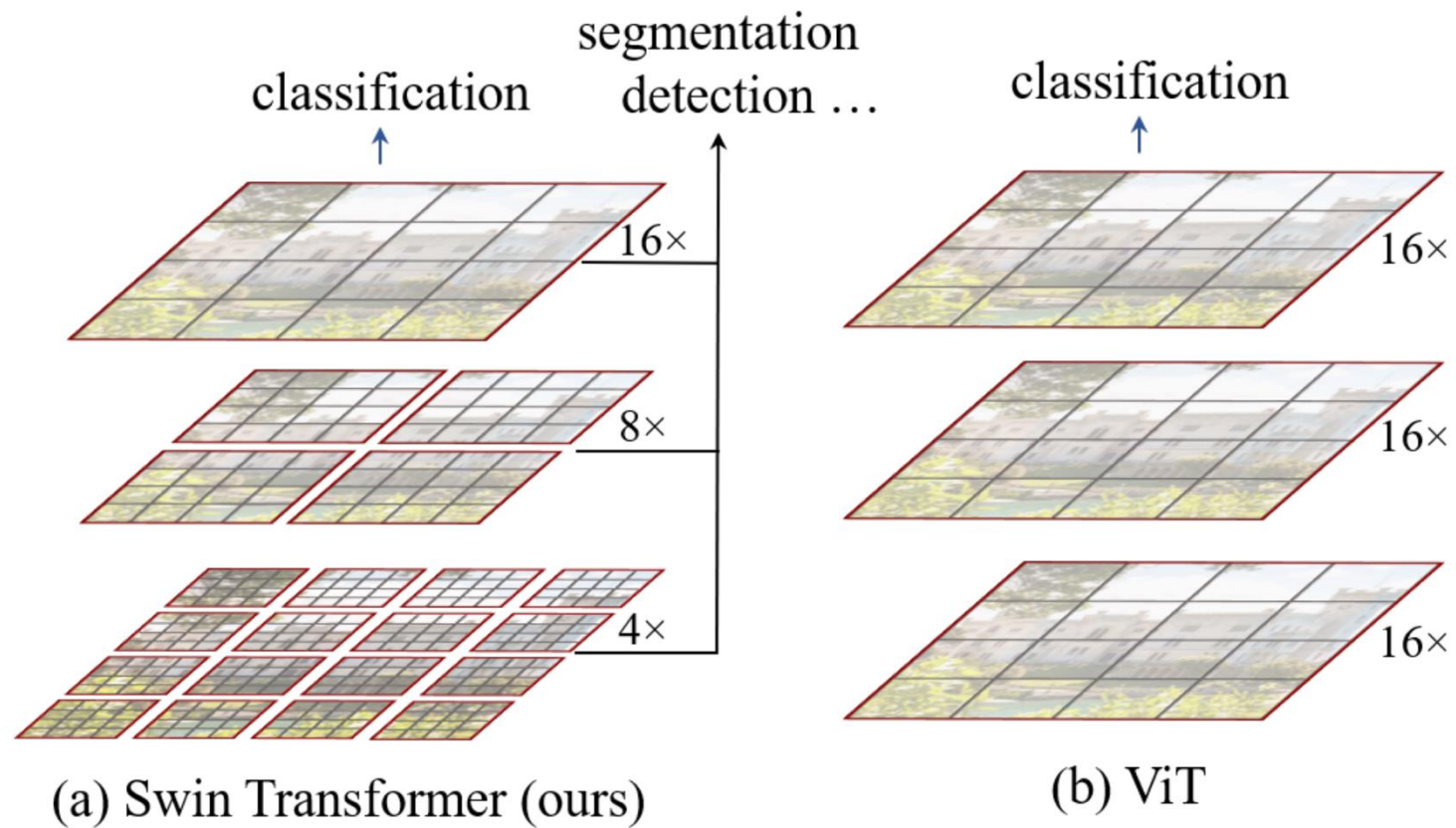
ViT do learn locality bias



Which one is better? Approximately the same.

Model	Epochs	ImageNet	ImageNet ReaL	CIFAR-10	CIFAR-100	Pets	Flowers	exaFLOPs
ViT-B/32	7	80.73	86.27	98.61	90.49	93.40	99.27	164
ViT-B/16	7	84.15	88.85	99.00	91.87	95.80	99.56	743
ViT-L/32	7	84.37	88.28	99.19	92.52	95.83	99.45	574
ViT-L/16	7	86.30	89.43	99.38	93.46	96.81	99.66	2586
ViT-L/16	14	87.12	89.99	99.38	94.04	97.11	99.56	5172
ViT-H/14	14	88.08	90.36	99.50	94.71	97.11	99.71	12826
ResNet50x1	7	77.54	84.56	97.67	86.07	91.11	94.26	150
ResNet50x2	7	82.12	87.94	98.29	89.20	93.43	97.02	592
ResNet101x1	7	80.67	87.07	98.48	89.17	94.08	95.95	285
ResNet152x1	7	81.88	87.96	98.82	90.22	94.17	96.94	427
ResNet152x2	7	84.97	89.69	99.06	92.05	95.37	98.62	1681
ResNet152x2	14	85.56	89.89	99.24	91.92	95.75	98.75	3362
ResNet200x3	14	87.22	90.15	99.34	93.53	96.32	99.04	10212
R50x1+ViT-B/32	7	84.90	89.15	99.01	92.24	95.75	99.46	315
R50x1+ViT-B/16	7	85.58	89.65	99.14	92.63	96.65	99.40	855
R50x1+ViT-L/32	7	85.68	89.04	99.24	92.93	96.97	99.43	725
R50x1+ViT-L/16	7	86.60	89.72	99.18	93.64	97.03	99.40	2704
R50x1+ViT-L/16	14	87.12	89.76	99.31	93.89	97.36	99.11	5165

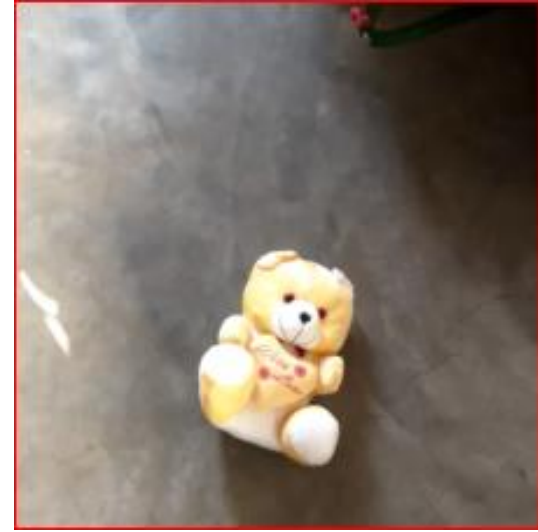
Hierarchical Vision Transformer



Issues with transformers consider, image classification



Tank



Teddy Bear

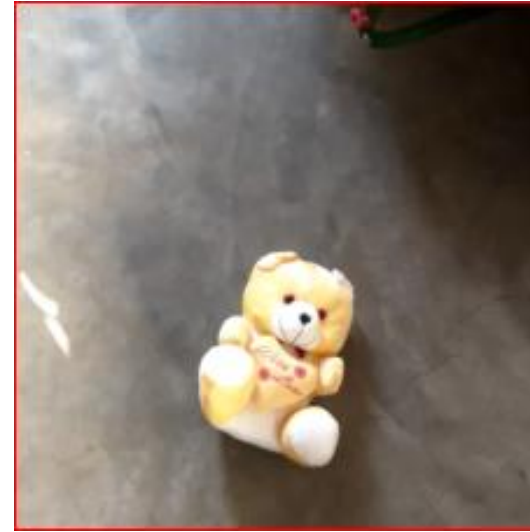
Spurious correlations

Background bias



Tram

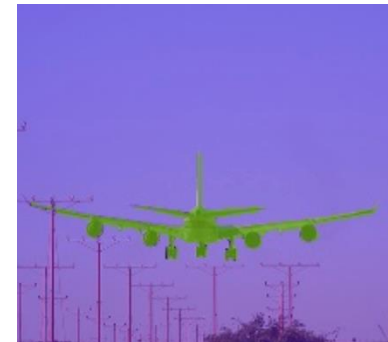
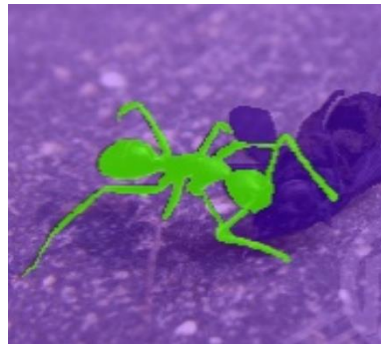
Partial perception



Ping-pong ball

Where the model should look

- In a classification task, we expect the model to perceive the **foreground**, not the **background**
- Segmentation maps from ImageNet-S:



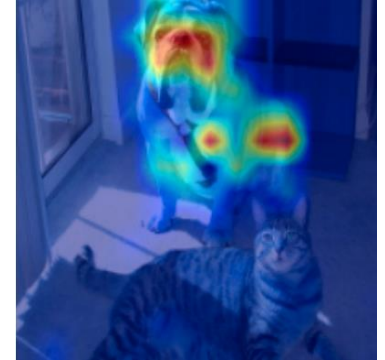
Where the model “looks”

- We estimate where the model looks with **Relevance map**
- It shows the parts of the input that influenced the decision of a model
- We'll consider a method that combines the attention maps and gradient signal*

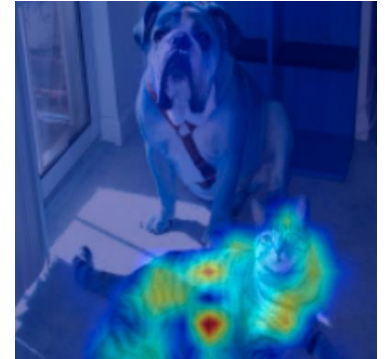
Global explanation
↑

Class specific
↑

Dog



Cat



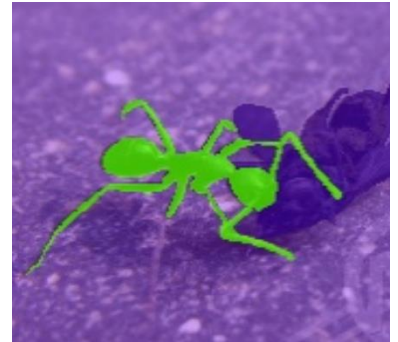
* Transformer interpretability beyond attention visualization (Chefer et al.; CVPR'21)

Optimize the relevance map

- Let $R(i)$ and $S(i)$ be the relevance map and segmentation for the image i

- The relevance map should be high on the **foreground**:

$$\mathcal{L}_{\text{fg}} = \text{MSE}(\mathbf{R}(i) \odot \mathbf{S}(i), 1)$$



- The relevance map should be low on the **background**:

$$\mathcal{L}_{\text{bg}} = \text{MSE}(\mathbf{R}(i) \odot \bar{\mathbf{S}}(i), 0)$$

- Combining the two loss terms:

$$\mathcal{L}_{\text{relevance}} = \lambda_{\text{bg}} \cdot \mathcal{L}_{\text{bg}} + \lambda_{\text{fg}} \cdot \mathcal{L}_{\text{fg}}$$

Calibration Hyperparameters

Keep the ability to classify objects

- Let $\mathcal{M}(i)$ be the model prediction for image i . We define a confidence-boost loss:

$$\mathcal{L}_{\text{classification}} = \text{CE}(\mathcal{M}(i), \arg \max(\mathcal{M}(i)))$$

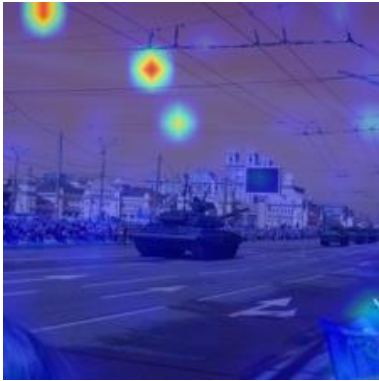
- Overall:

$$\mathcal{L} = \lambda_{\text{relevance}} \cdot \mathcal{L}_{\text{relevance}} + \lambda_{\text{classification}} \cdot \mathcal{L}_{\text{classification}}$$

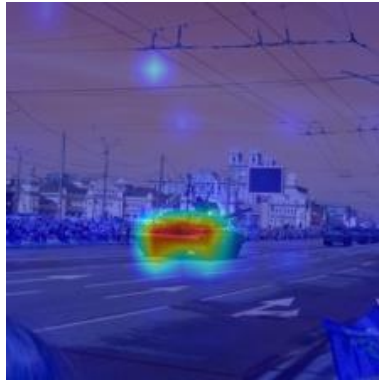
- Our optimization uses **a short finetuning** on pre-trained models on ImageNet

Corrected predictions

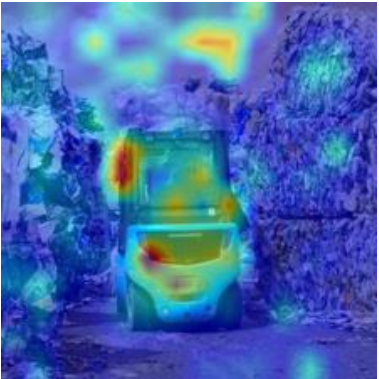
Background bias



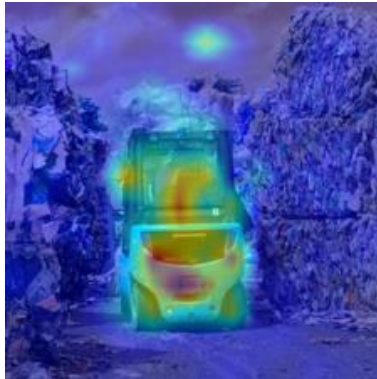
Tram



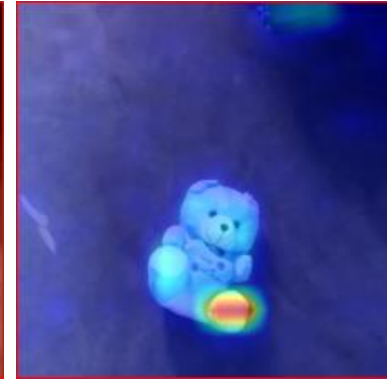
Tank



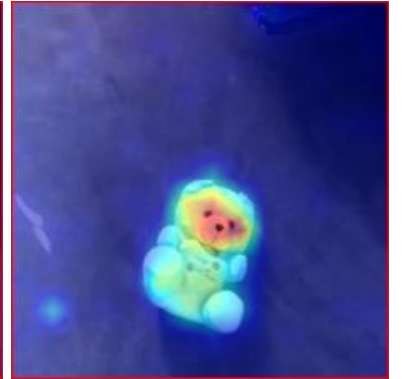
Garbage-truck



Forklift



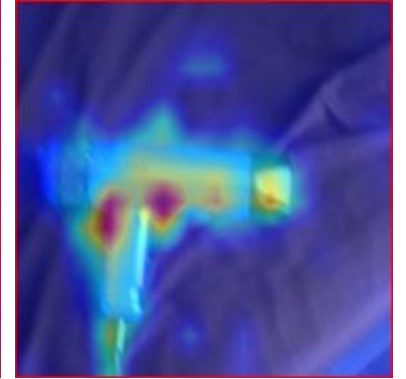
Ping-pong ball



Teddy

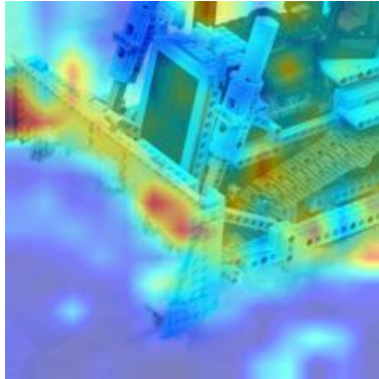
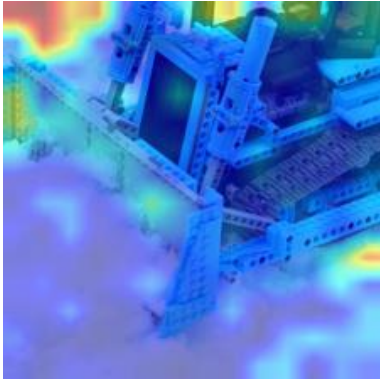


Drill



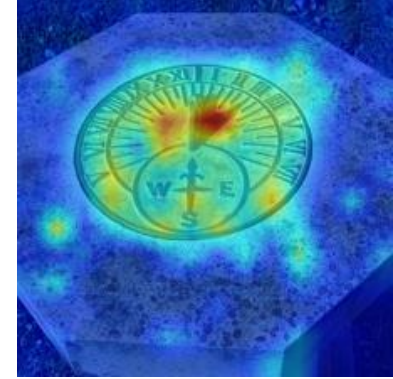
Hand-blower

Same prediction – but for the right reasons



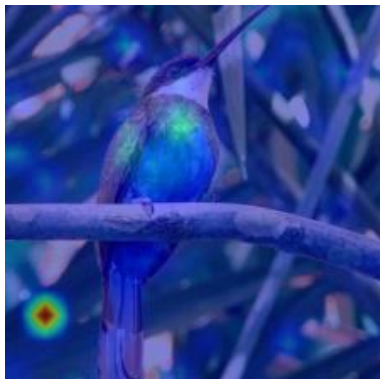
Snowplow

Snowplow



Sundial

Sundial



Jacamar

Jacamar



Curly coated
retriever

Curly coated
retriever

Incorrect prediction - reveals issues in the dataset

Multiple objects



Tripod



Strawberry



breastplate



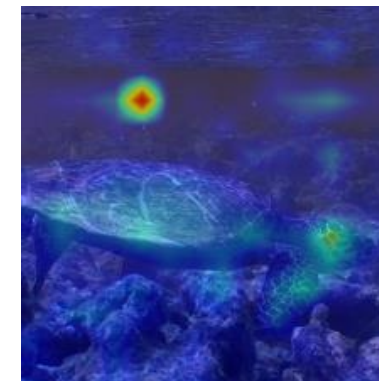
T-shirt



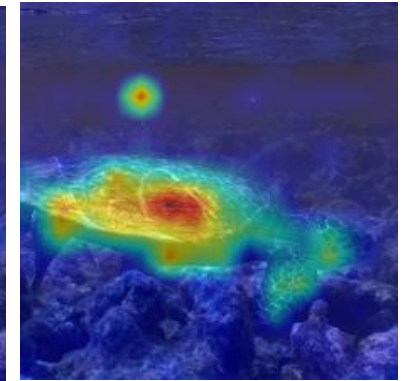
Pool-table



Hat



Coralreef



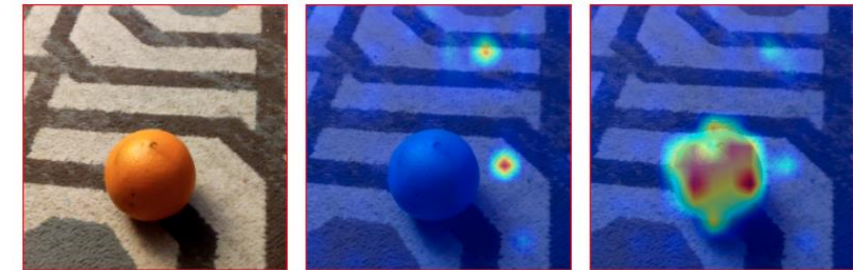
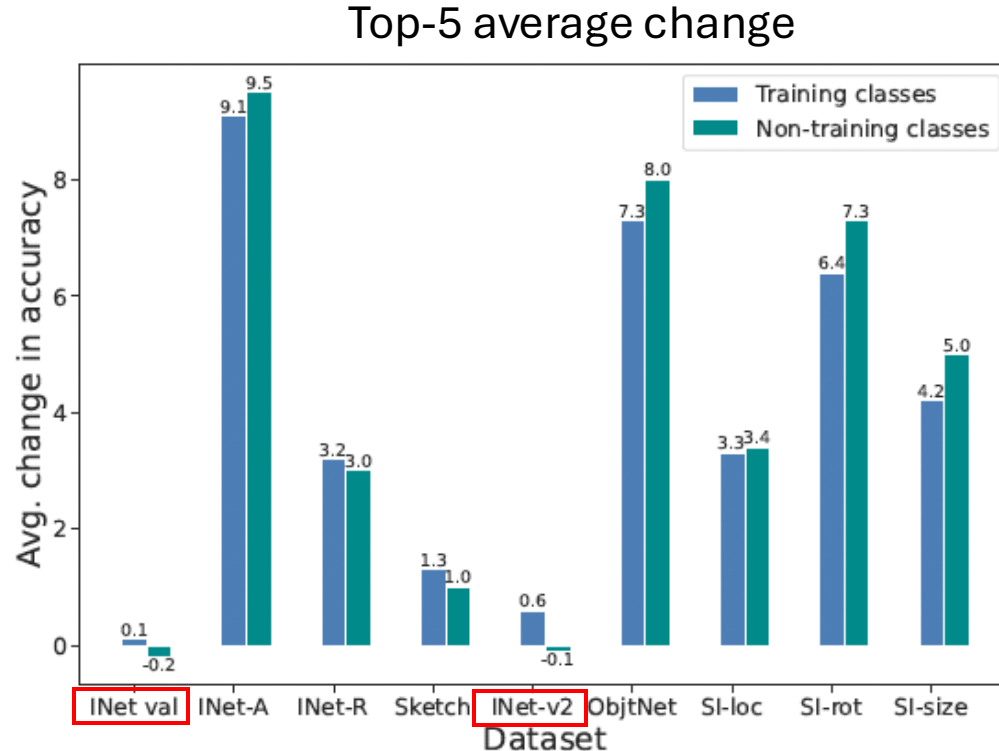
Leatherback Turtle

Benefits: Efficient

- We only use **3 samples per class**
- We only use half of the ImageNet classes, which make up 500 classes
- Overall, only 1500 samples
- Works even with DiNO, an unsupervised segmentation method

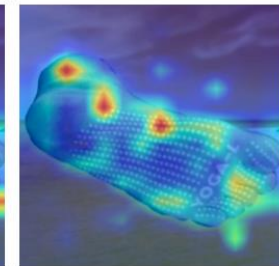
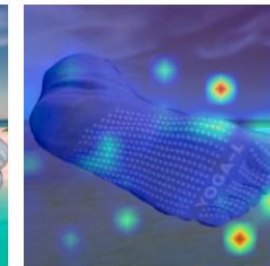
Benefits: Robust

- The method improves performance on unseen classes and datasets with shifted distribution from ImageNet



Maze

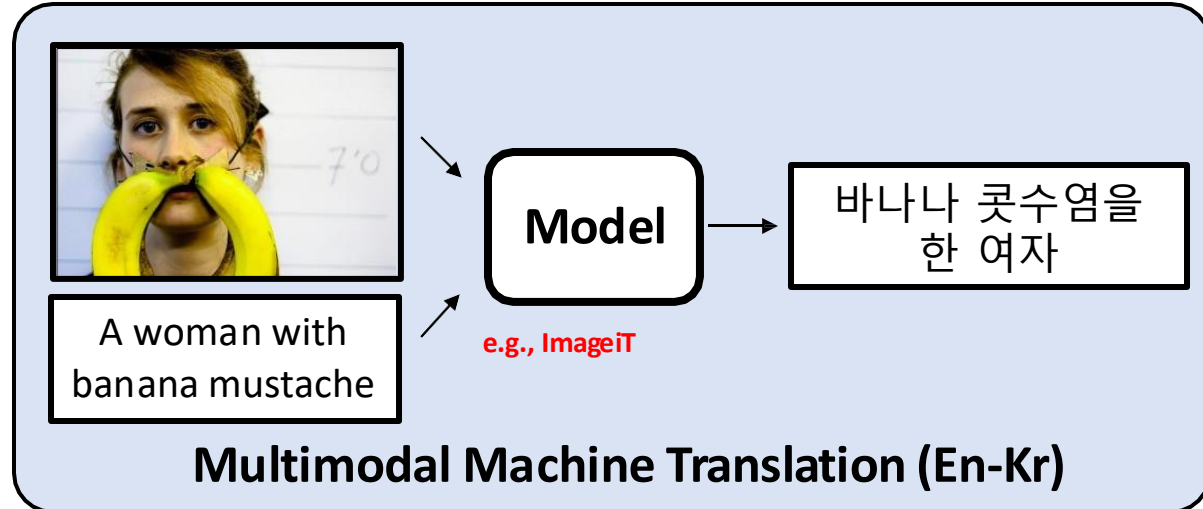
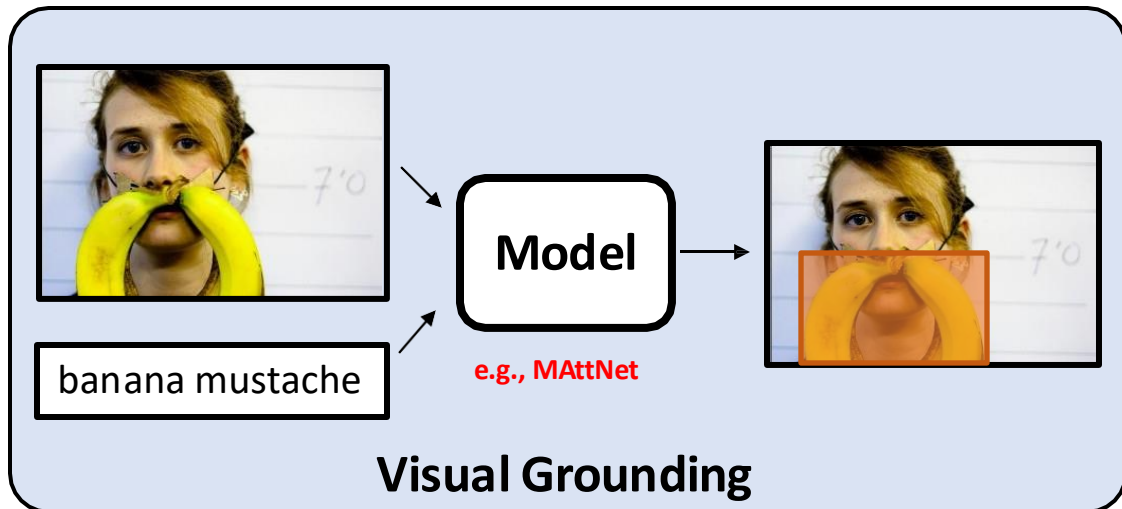
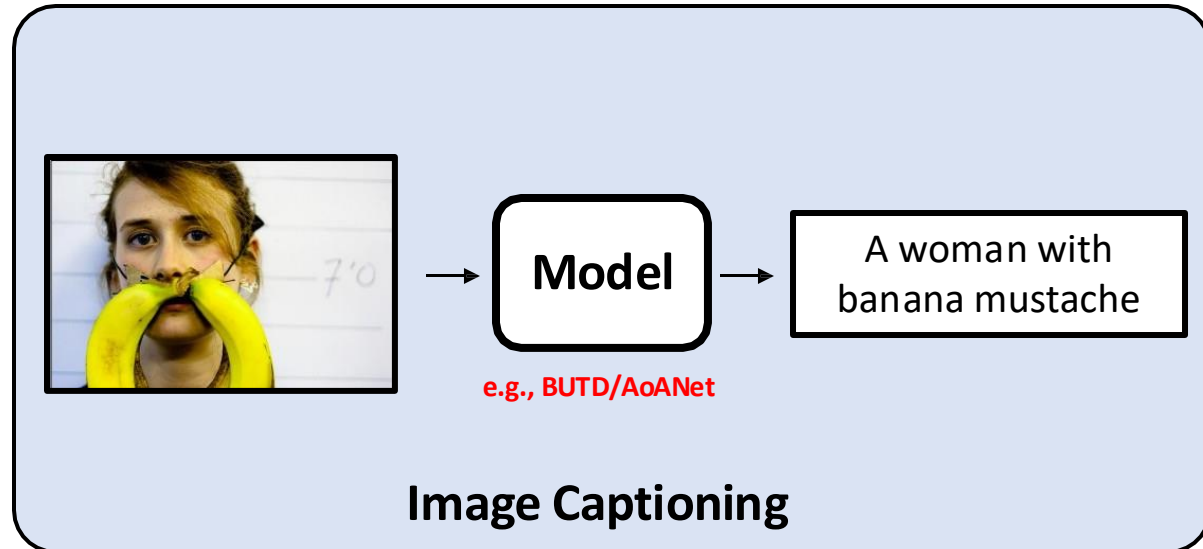
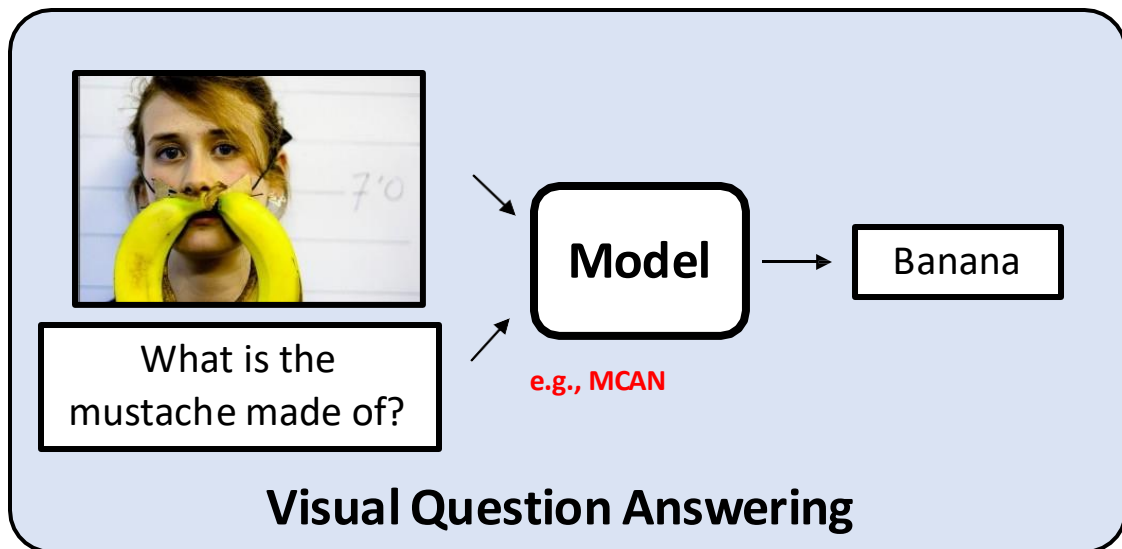
Orange



Swimmi-
ng trunks

Sock

What about multimodal?



Huang et al., 2019, "Attention on Attention for Image Captioning"

Yu et al., 2018, "MAAttNet: Modular Attention Network for Referring Expression Comprehension"

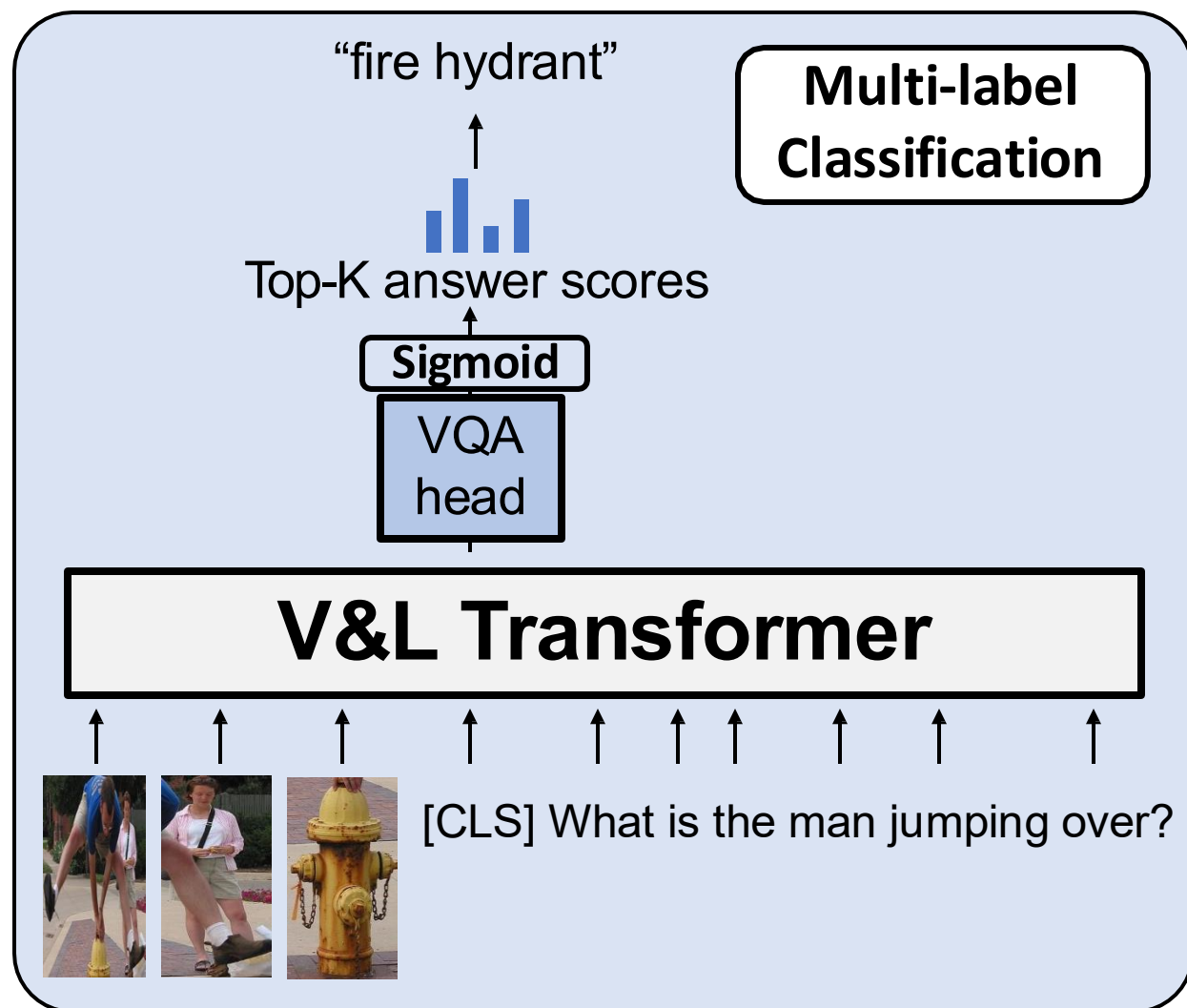
Long et al., 2021, "Generative Imagination Elevates Machine Translation"

Anderson et al., 2018, "Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering"

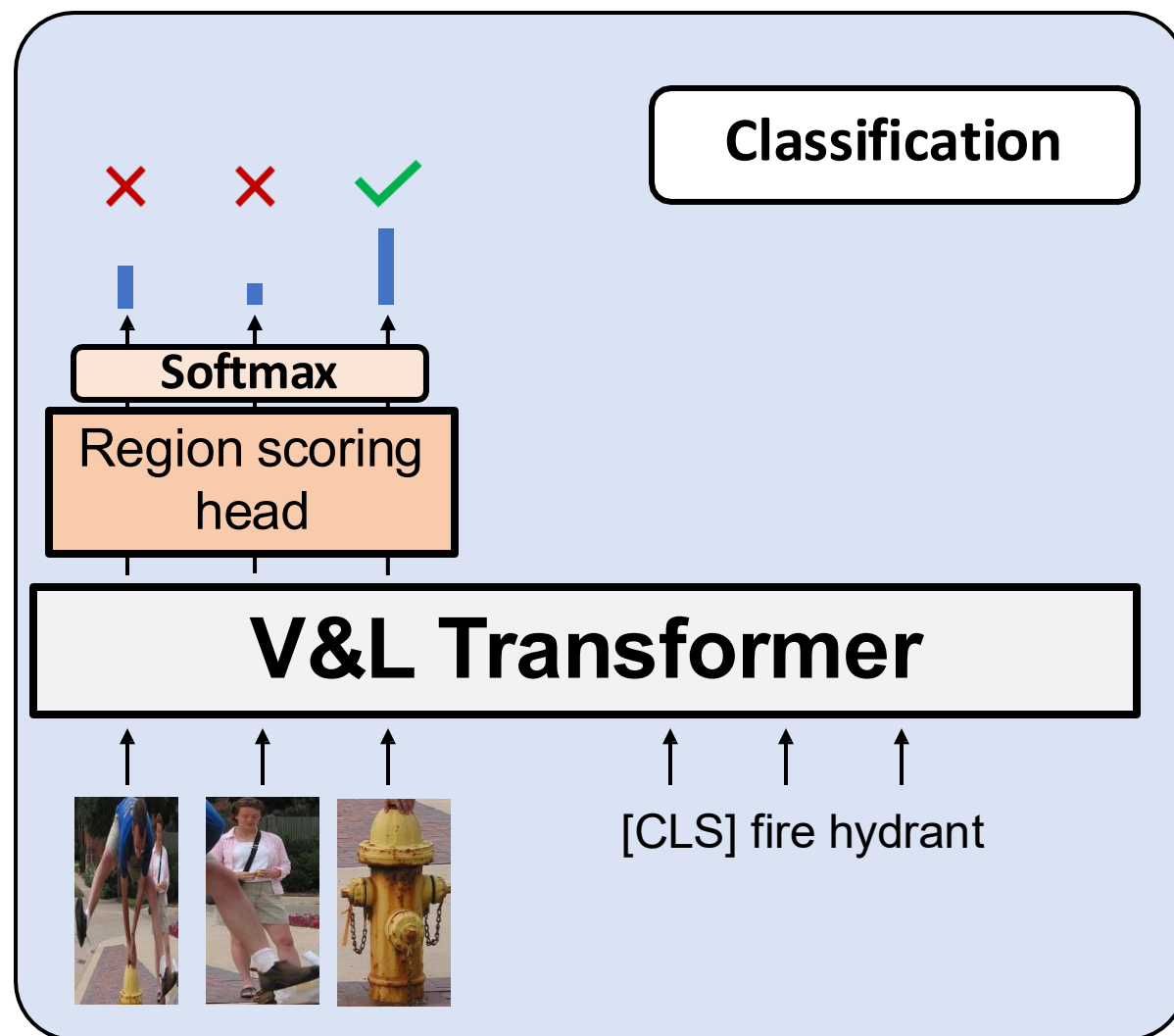
Yu et al., 2019, "Deep Modular Co-Attention Networks for Visual Question Answering"

Generalize with Transformers?

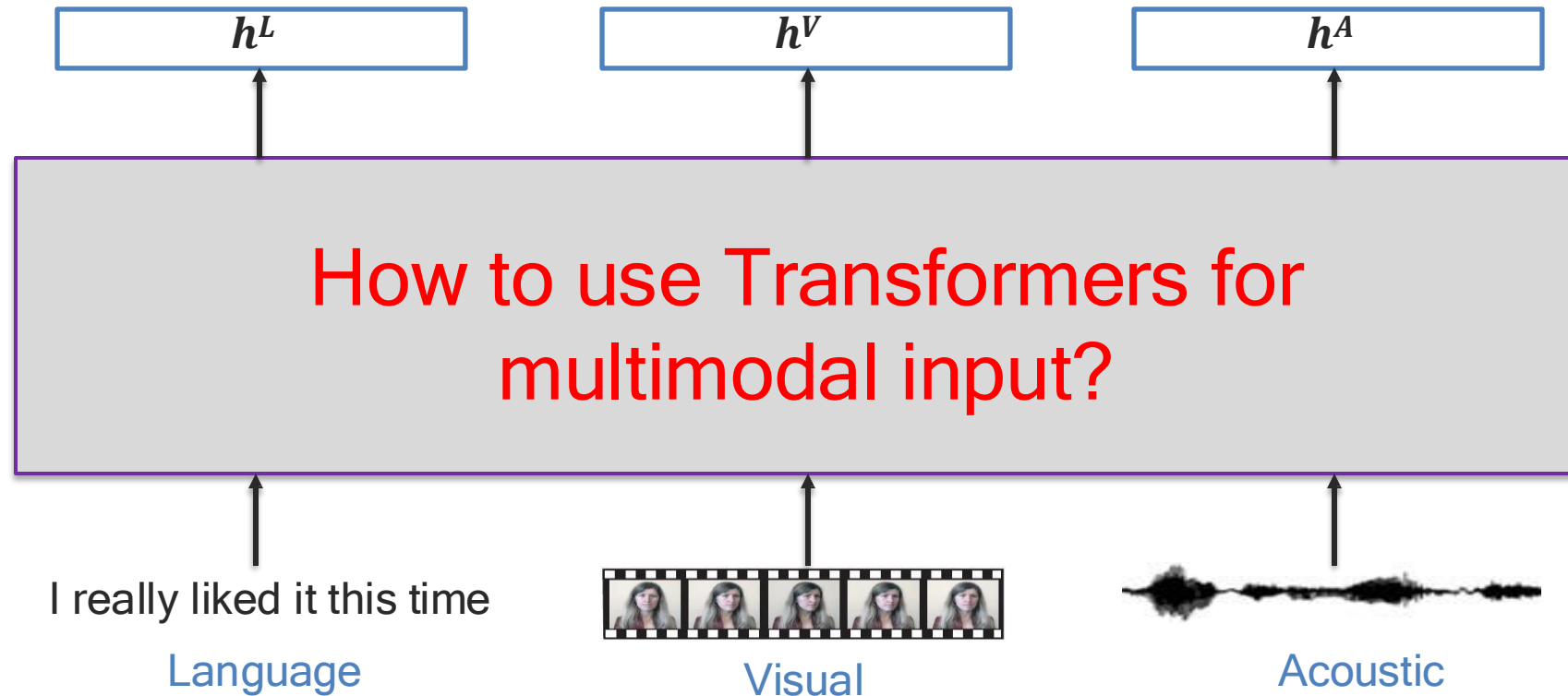
Visual Question Answering



Visual Grounding



Multimodal Transformers

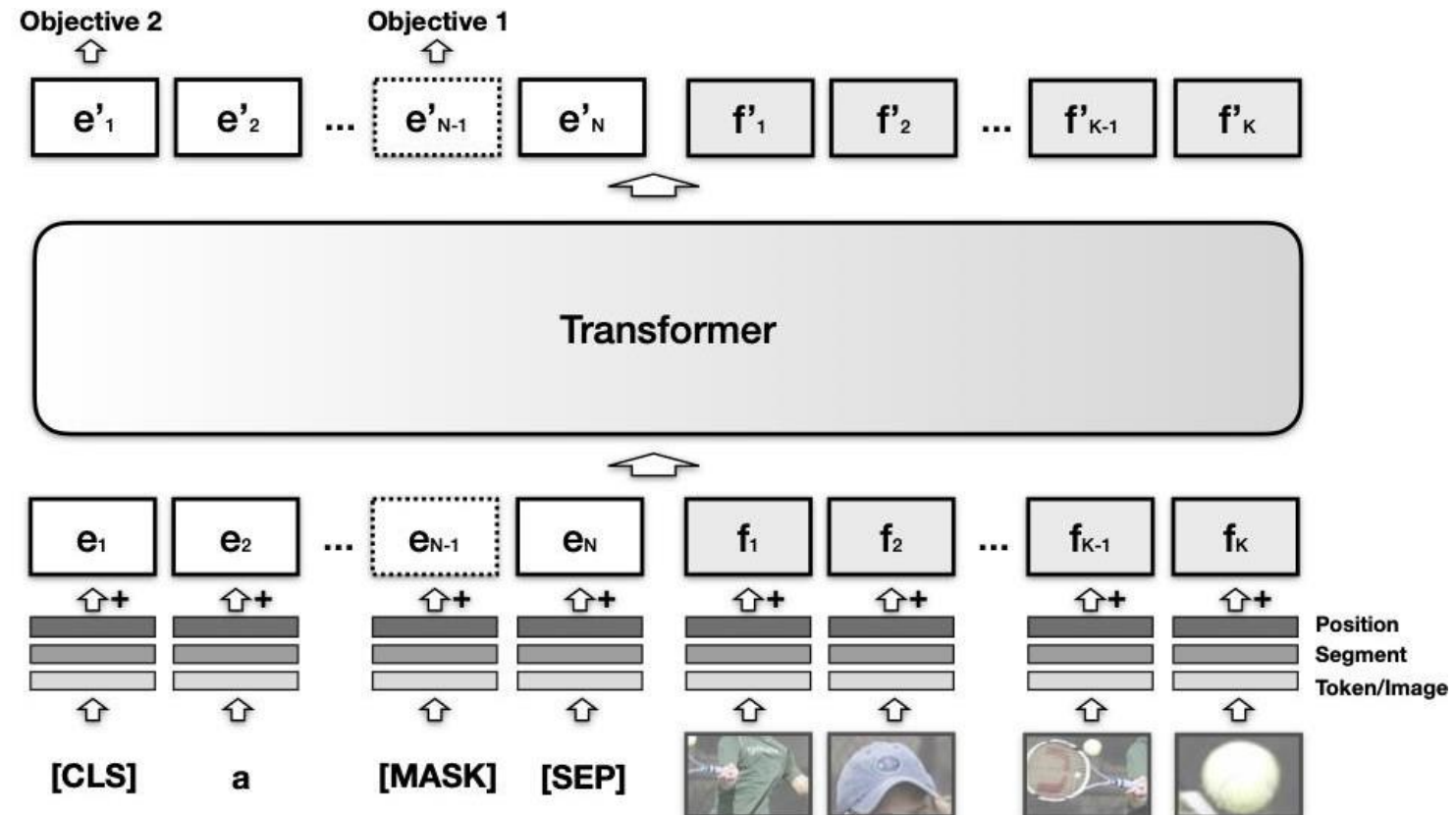


Option 1: Concatenate modalities and learn BERT transformer

VisualBERT



A person hits a ball with a tennis racket



Li, Liunian Harold, et al. "Visualbert: A simple and performant baseline for vision and language. (2019)

Option 2:

Recall, Cross-Attention

- Given two modalities $a \in \mathbb{R}^{n_A \times d}$, $b \in \mathbb{R}^{n_B \times d}$
- The attention score is based on a **dot-product**
 - $Q_A = a \cdot W_{A,Q} \in \mathbb{R}^{n_A \times d}$
 - $K_B = b \cdot W_{B,K} \in \mathbb{R}^{n_B \times d}$
 - Where: $W_{A,Q} \in \mathbb{R}^{d \times d}$, $W_{B,K} \in \mathbb{R}^{d \times d}$

Correlation Matrix:

$$\begin{matrix} & Q_A \\ \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \begin{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \end{matrix} \\ \times \\ \begin{matrix} & K_B^T \end{matrix} \end{matrix} \in \mathbb{R}^{n_A \times n_B}$$

Cross-attention correlation matrix

$$Q_A \times K_B^T \in \mathbb{R}^{n_A \times n_B}$$


Correlation Matrix:

			
What	1.2	2.3	1.7
animal	-0.4	6.5	4.2
is	1.3	1.2	0.6

Cross-attention for multimodal

1)

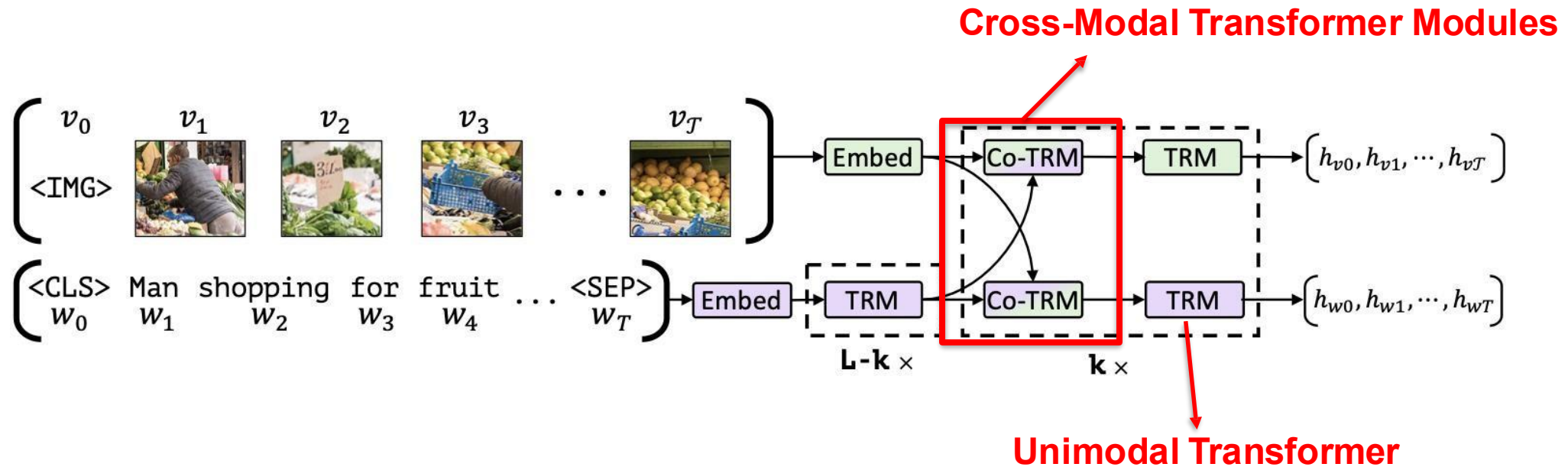
$$\begin{aligned}
 & \text{softmax} \left(\frac{\overbrace{Q_A \times K_B^T}^{\mathbb{R}^{n_A \times n_B}}}{\sqrt{d_k}} \right) V_B \\
 & \quad \mathbb{R}^{n_B \times d} \\
 & \quad V_B \\
 & = Z_{B \rightarrow A} \\
 & \quad \mathbb{R}^{n_A \times d}
 \end{aligned}$$

2)

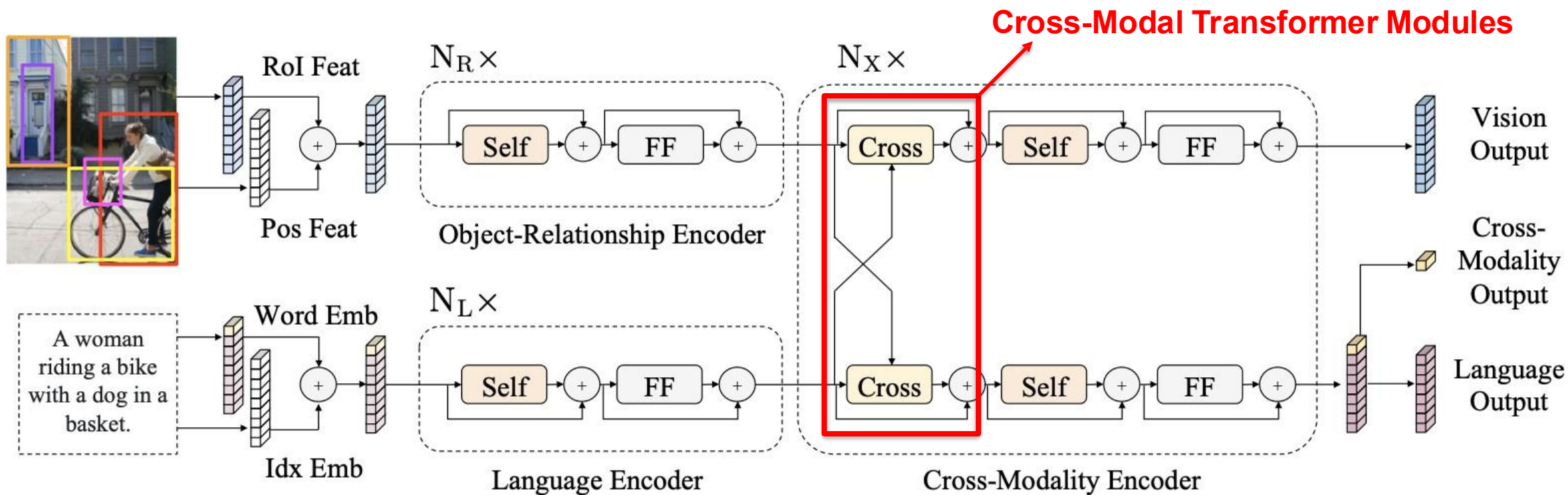
Residual connection

$$Q_A + Z_{B \rightarrow A}$$

ViLBERT



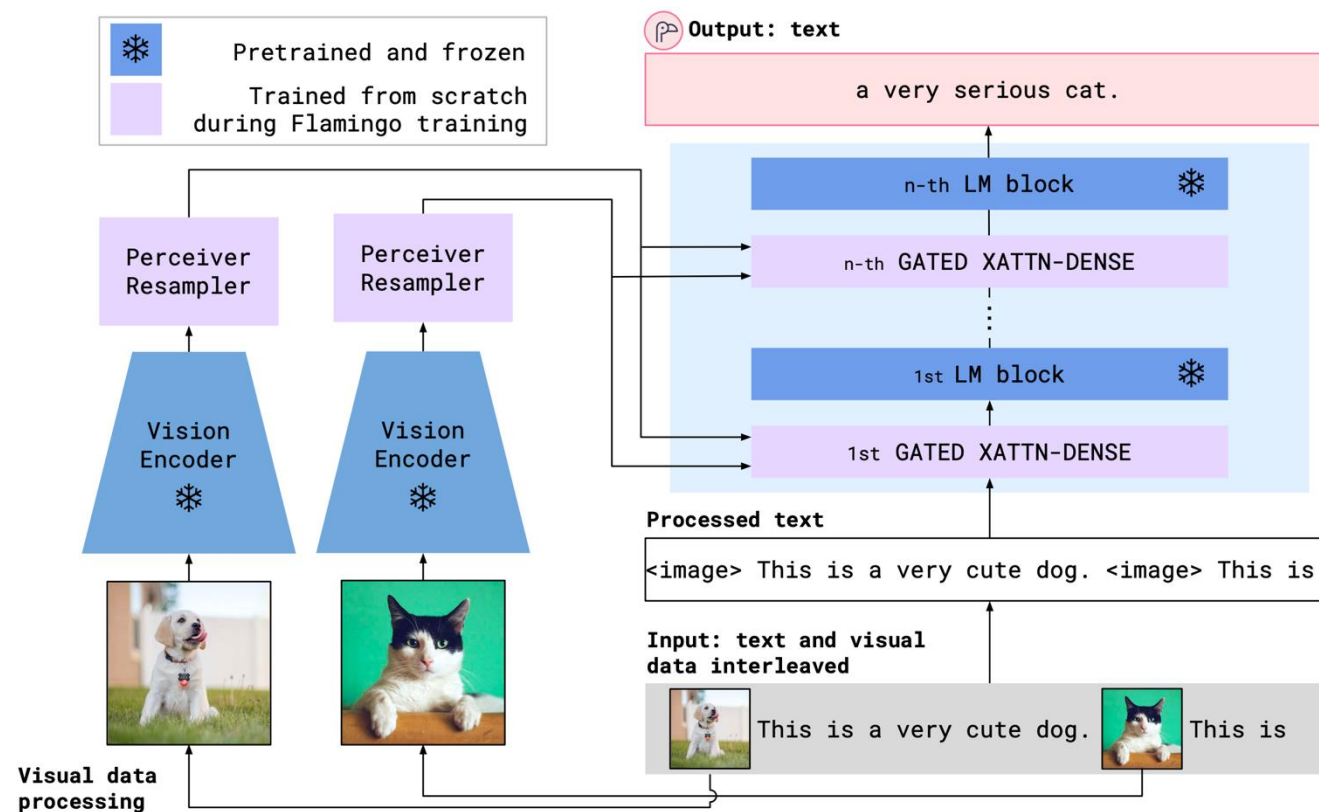
LXMERT



Tan, Hao, and Mohit Bansal. "Lxmert: Learning cross-modality encoder representations from transformers." *arXiv* (August 20, 2019).

Flamingo

- Employ 70B Frozen LLM
- Large-scale dataset: 43M web pages, Multimodal MassiveWeb (M3W)



How to get to GPT-4?

Instruction data

Context type 1: Captions

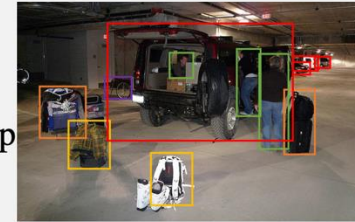
A group of people standing outside of a black vehicle with various luggage.

Luggage surrounds a vehicle in an underground parking area

People try to fit all of their luggage in an SUV.

The sport utility vehicle is parked in the public garage, being packed for a trip

Some people with luggage near a van that is transporting it.



Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], backpack: [0.384, 0.696, 0.485, 0.914], suitcase: ...<omitted>

Response type 1: conversation

Question: What type of vehicle is featured in the image?

Answer: The image features a black sport utility vehicle (SUV) ...<omitted>

Response type 2: detailed description

The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip. ...<omitted>

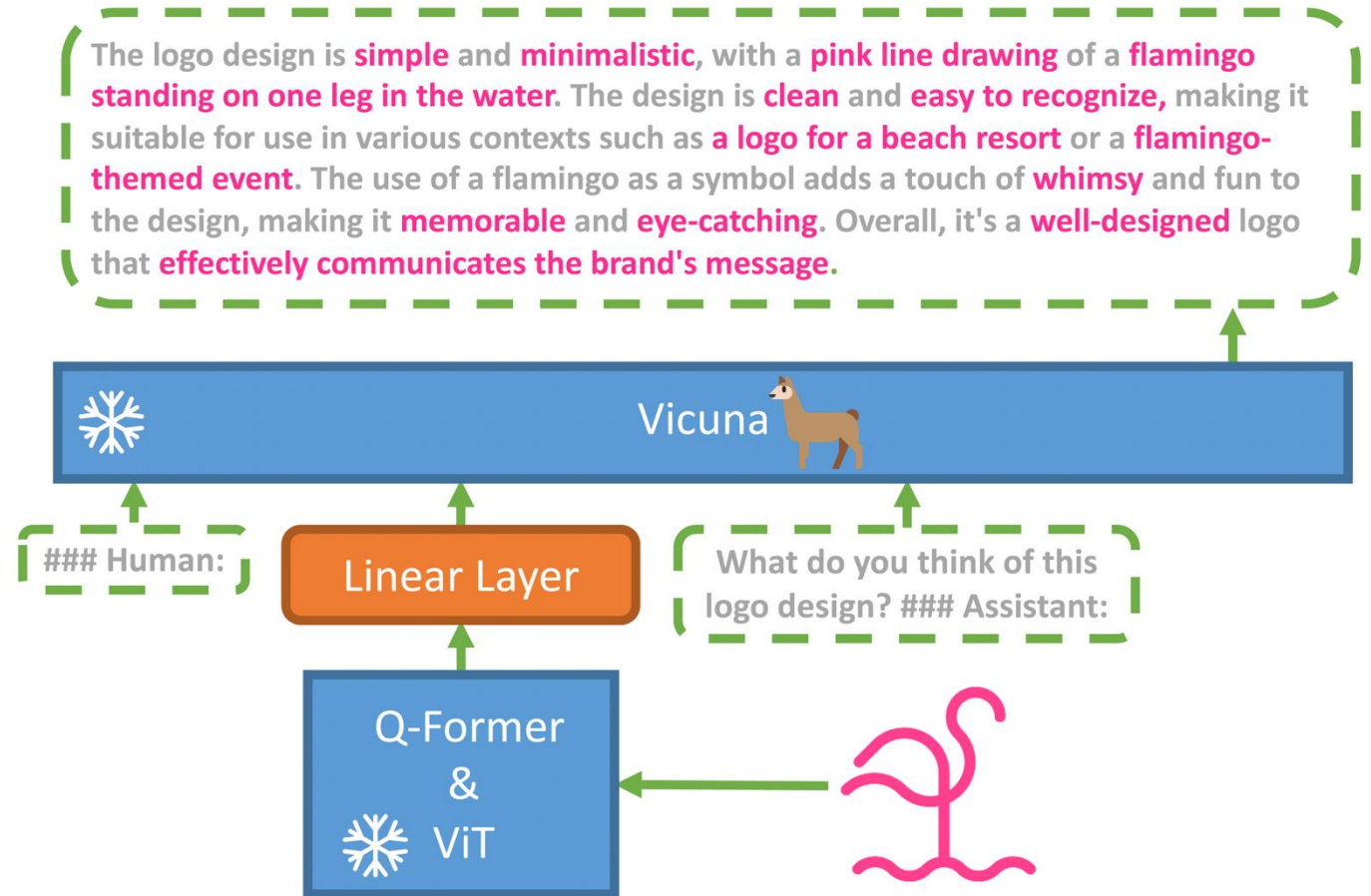
Response type 3: complex reasoning

Question: What challenges do these people face?

Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases and backpacks to be packed, which suggests that the group has a significant amount of belongings ...<omitted>

Mini-GPT-4

- Stronger LLM (Vicuna)
- Instruction tuning



Qualitative Example



What is unusual about this image?

Responses

BLIP-2: a man is sitting on the back of a yellow cab

OpenFlamingo: The man is drying his clothes on the hood of his car

GPT-4: The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.

BLIP and Flamingo supervision lead to descriptive responses instead of answering the instruction.

Basic steps for GPT-4 like model

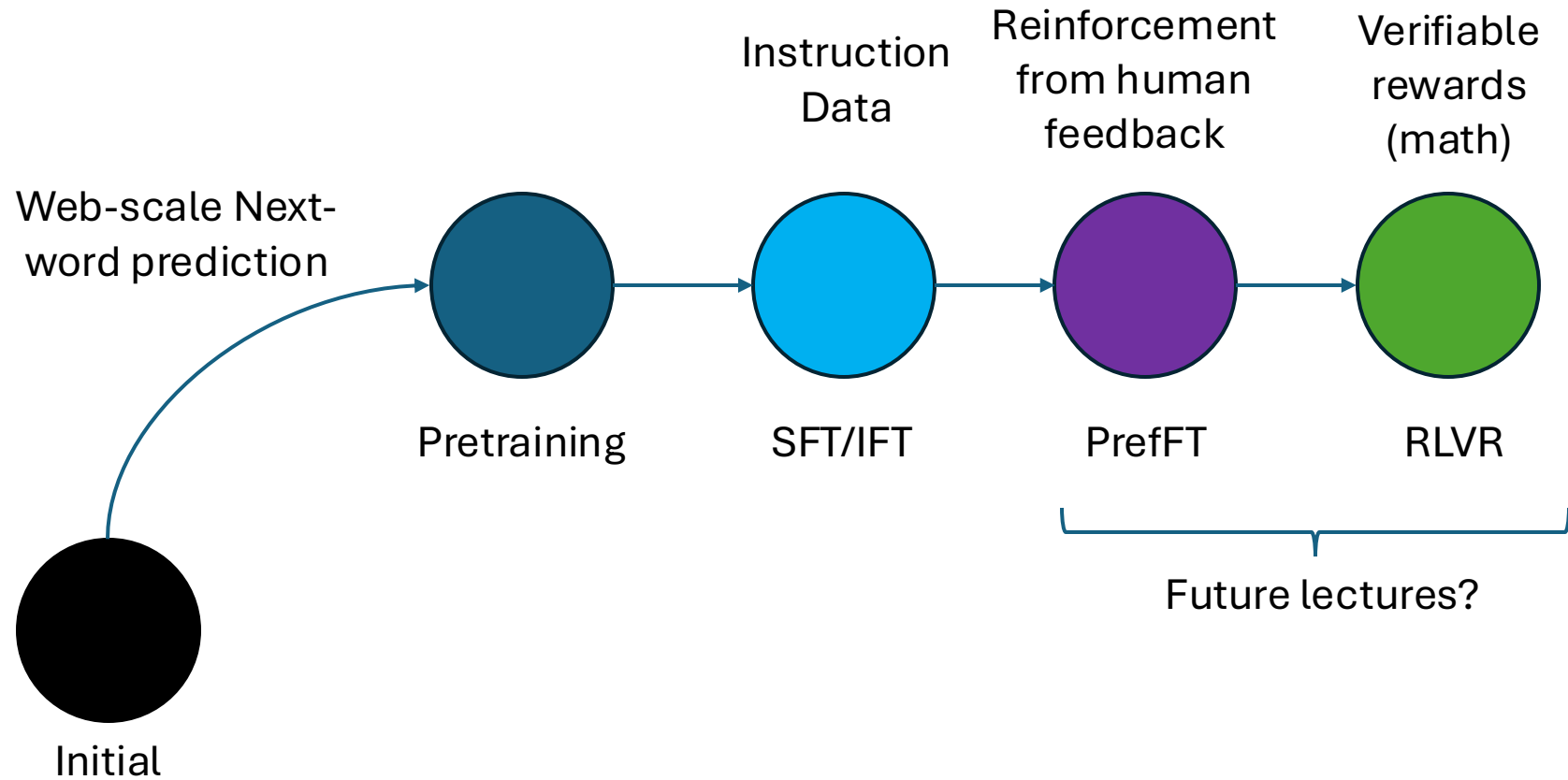


Image Generation Models (Part 1)

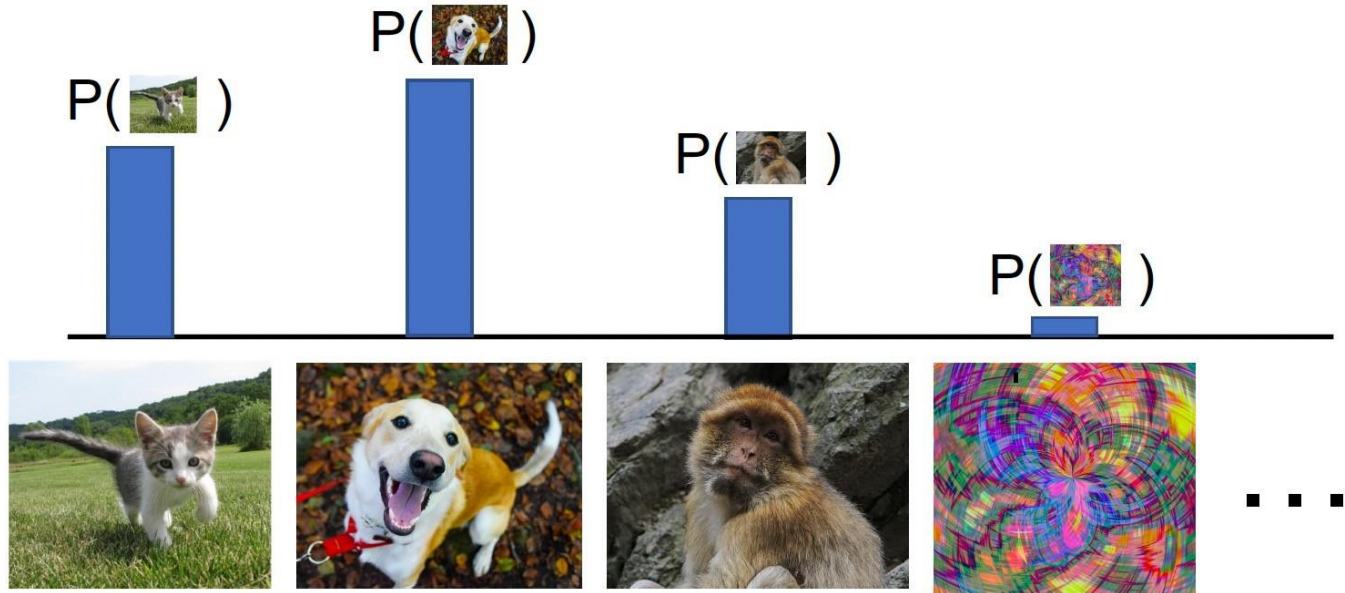
Discriminative vs Generative models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

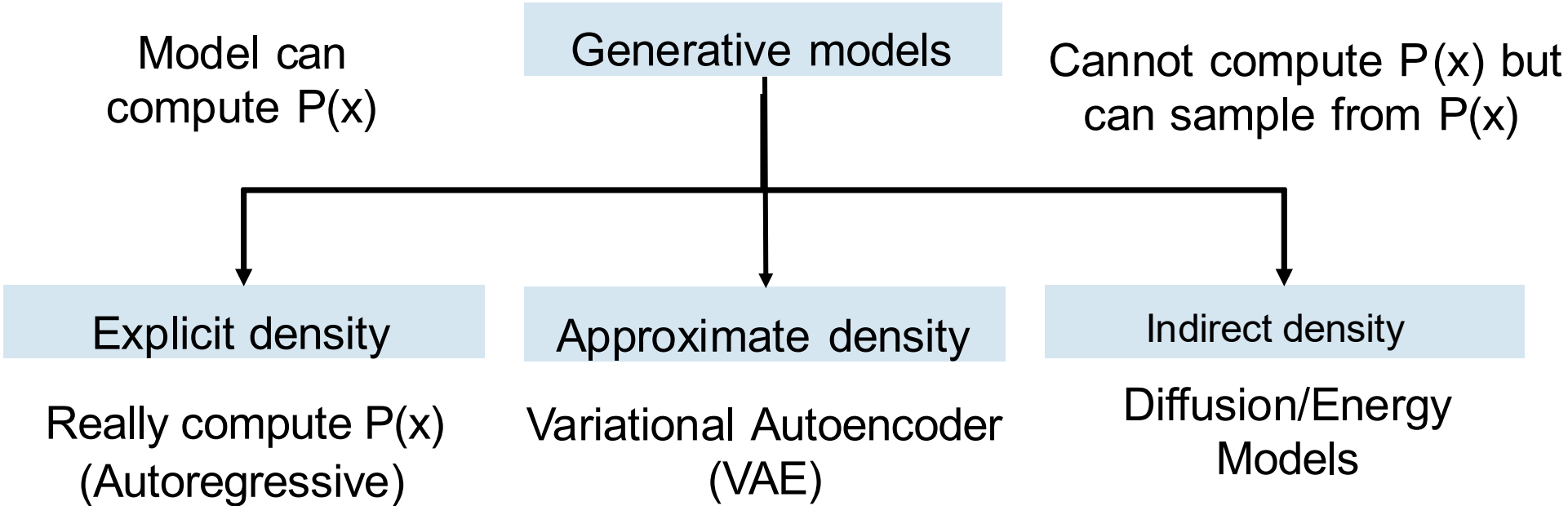


- Generative model: All possible images compete with each other for probability mass
- Model can “reject” unreasonable inputs by assigning them small values

Generative Modeling

- The goal of generative modeling - learning a model of a distribution from which we can generate new samples.
- Given $x \sim p(x)$ we want to learn $p_{\theta}(x) \approx p(x)$ (modeling)
- Then, we can generate new samples $x \sim p_{\theta}(x)$ (generation)

Taxonomy of Generative Models



Explicit density model

$$p(x) = p(x_1, x_2, \dots, x_n)$$

Likelihood of
image x

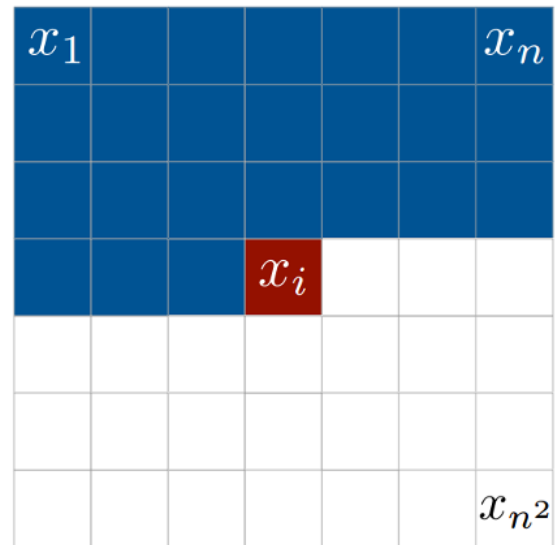
Joint likelihood of each
pixel in the image

Autoregressive network

Use chain rule to decompose likelihood of an image x into product of 1-d distributions:

$$p(x) = p(x_1, x_2, \dots, x_{n^2}) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \dots$$

Express using a sequence neural network!



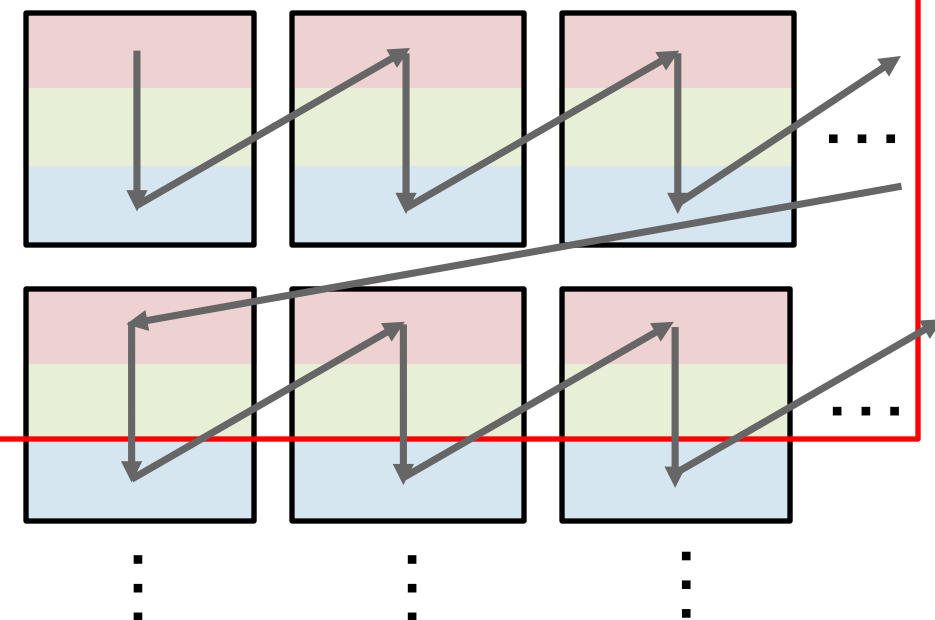
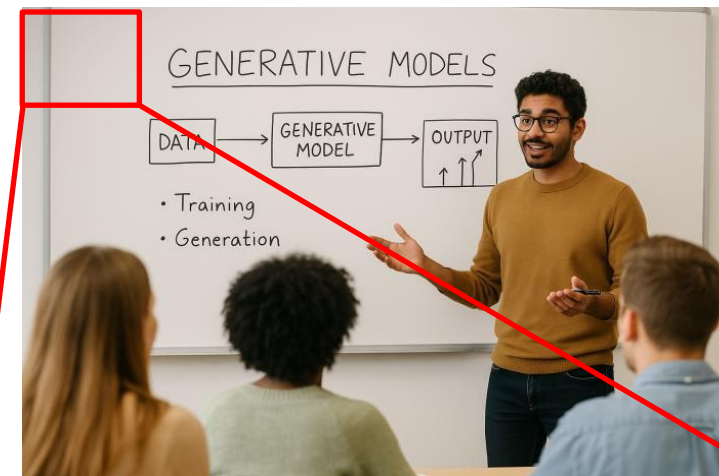
Autoregressive Models of Images

Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

Model with an RNN or Transformer

Problem: Too expensive. 1024×1024 image is a sequence of 3M subpixels



Variational Autoencoders

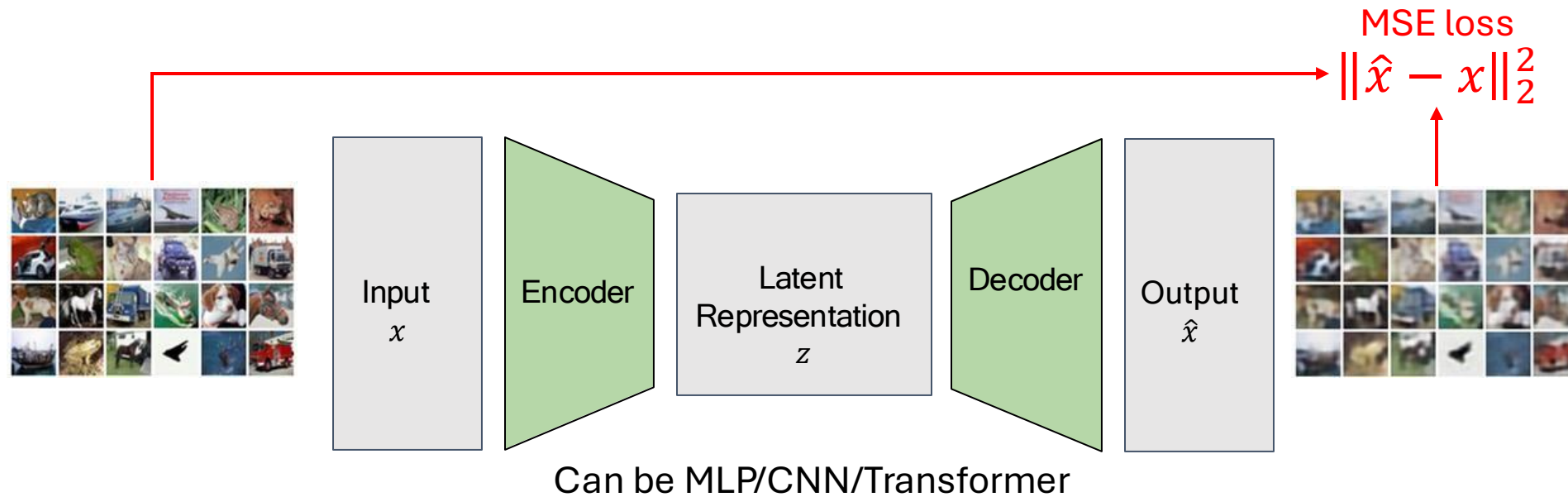
- PixelRNN explicitly parameterizes density function with a neural network, so we can train to maximize likelihood of training data:

$$p_W(x) = \prod_{t=1}^T p_W(x_t \mid x_1, \dots, x_{t-1})$$

- Variational Autoencoders (VAE) define an **intractable density** that we cannot explicitly compute or optimize
- But we will be able to directly optimize a **lower bound** on the density

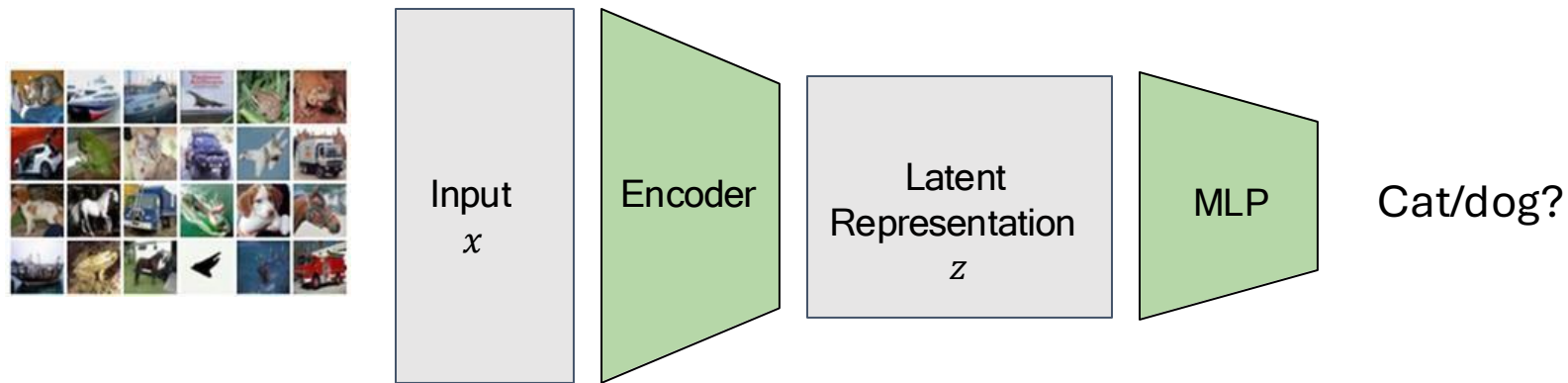
(Non-Variational) Autoencoders

- **Idea:** learning to extract features z from inputs x , without labels



(Non-Variational) Autoencoders

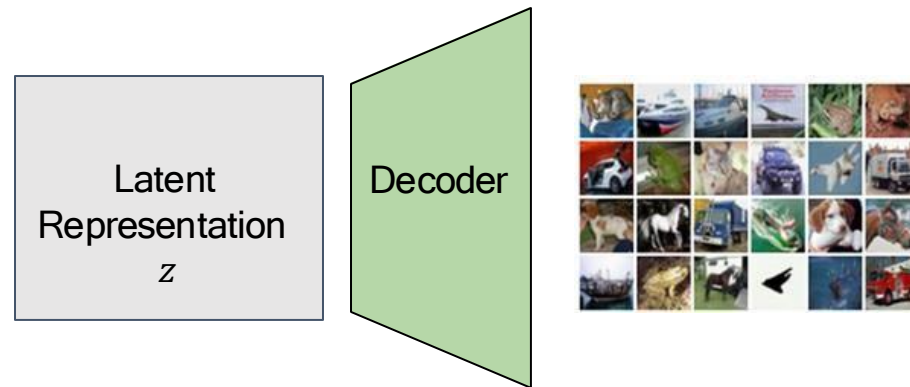
- After training, can use encoder for downstream tasks



- This is a self supervised approach
- Masked Autoencoder (MAE) is an extension of Autoencoder

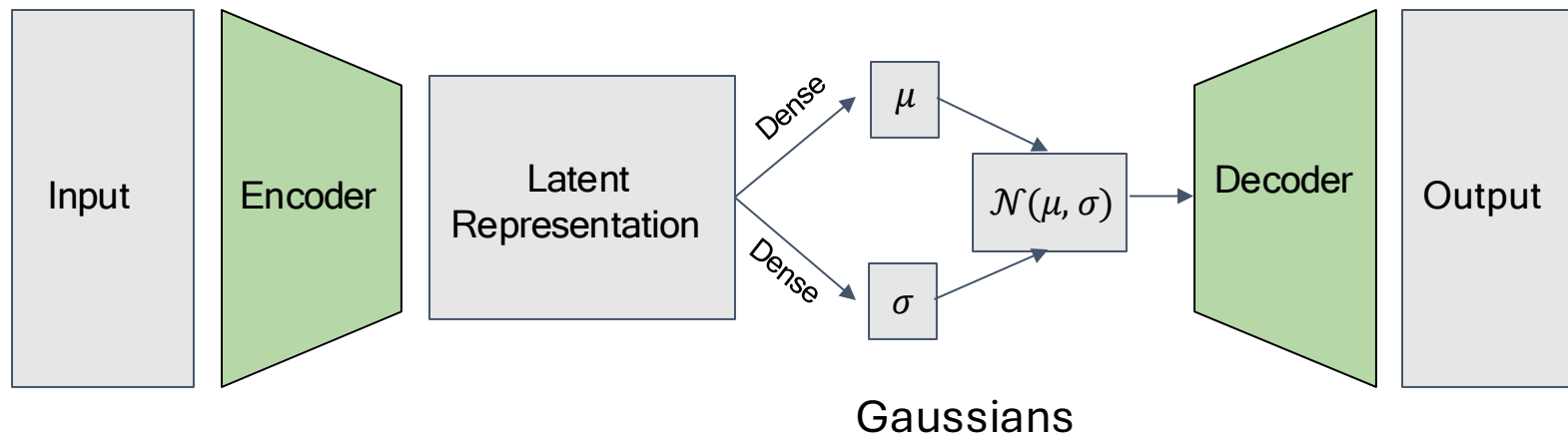
(Non-Variational) Autoencoders

- What if we wanted to generate images at inference?



- Generating z that will be decoded to an image, is not any easier than generation an image.
- Solution: force all z to come from a known distribution

Variational Auto-encoder



Weighted Combination of Losses

L_1 = loss associated with producing output similar to input

L_2 = loss associated with producing output with some variation to input

Total Loss: $L = L_1 + \lambda L_2$

$\lambda \in [0, \infty]$



- Also motivated from Evidence lower bound (ELBO)*

* - https://en.wikipedia.org/wiki/Evidence_lower_bound

VAE Losses, Defined

We have seen L_1 before: this is just the autoencoder reconstruction loss

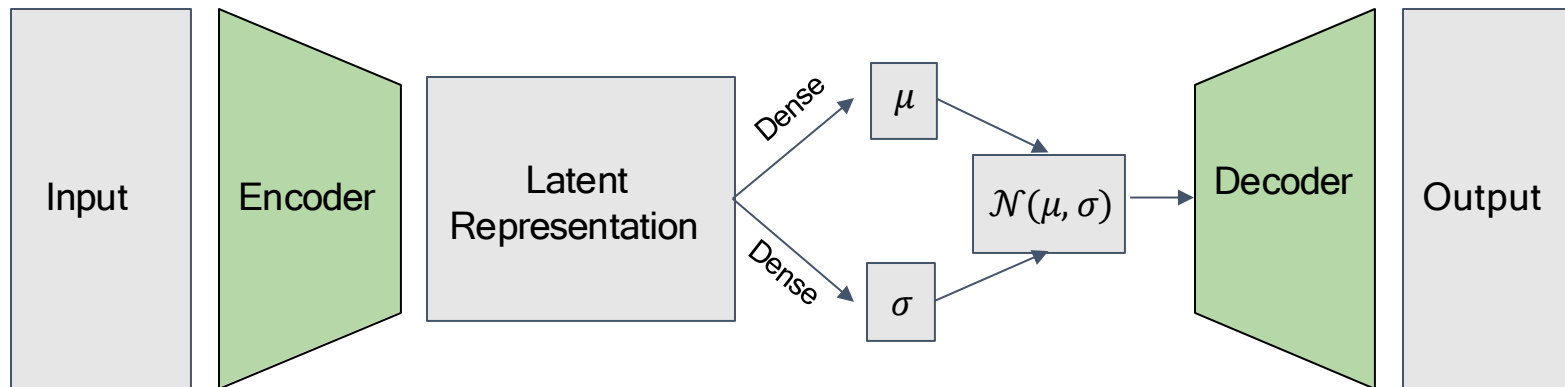
$$L_1(x, \hat{x}) = ||x - \hat{x}||_2^2$$

But with L_2 , it's not so clear. How do we measure how much variation our output would have?

$$L_2(??, ??) = ??$$

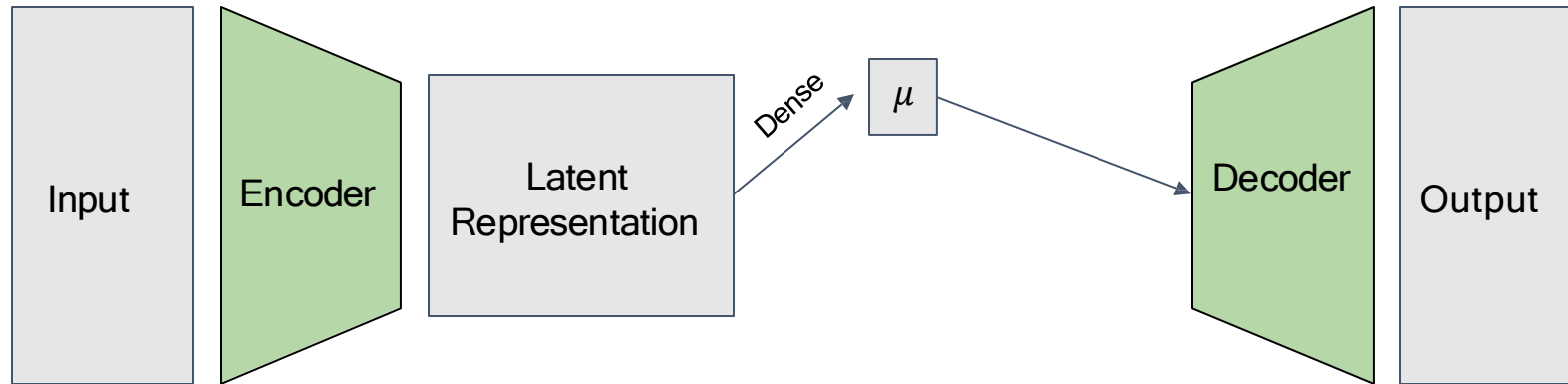
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions



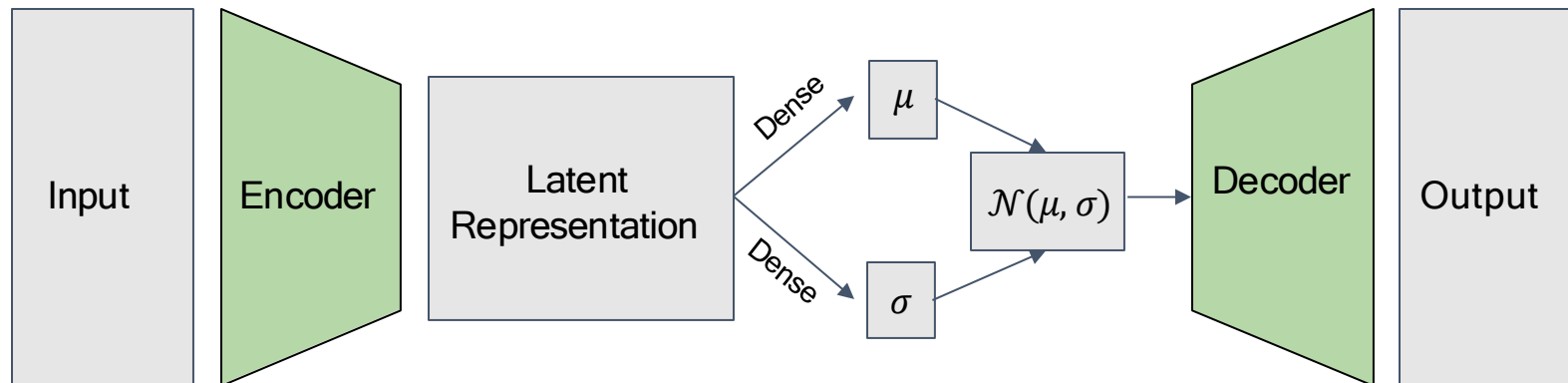
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
 - Behaves the same as a regular autoencoder!



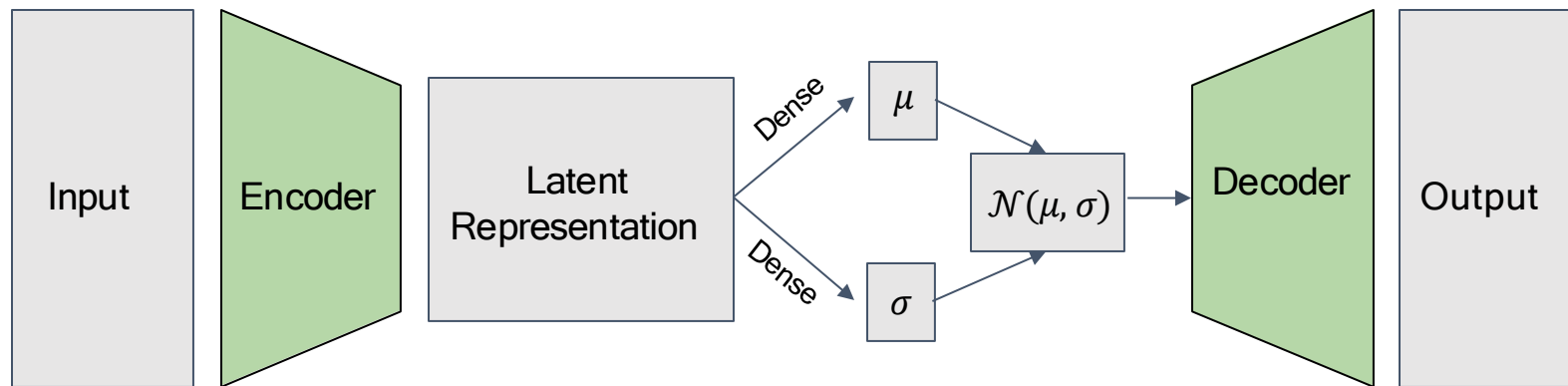
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
- But how big should we encourage σ to be?
- And for that matter, what we do about μ ?



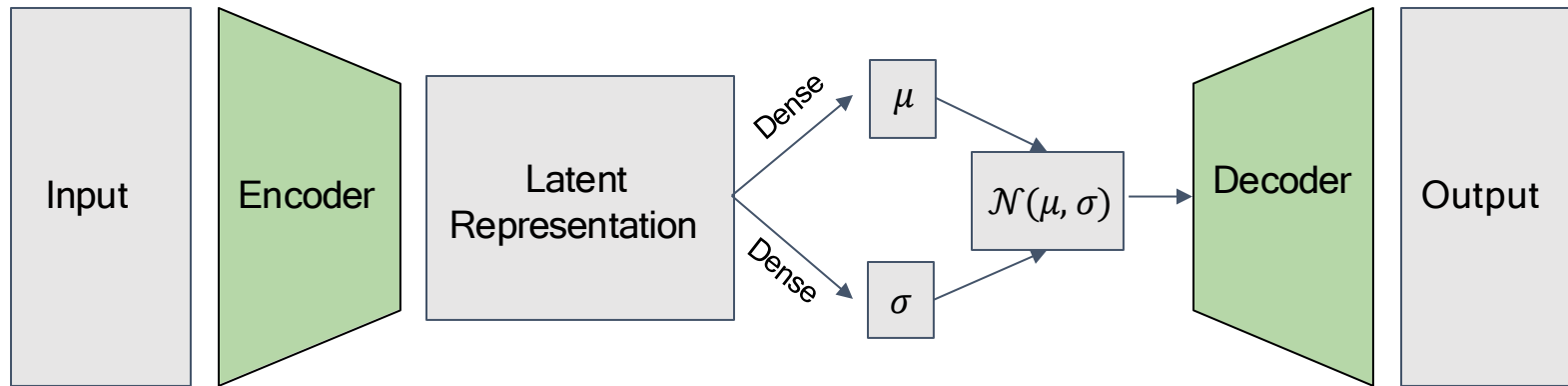
Defining the L_2 Loss...

- **The idea:** make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$
 - we'll see later that having some light pressure to make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$ will have some beneficial properties



Defining the L_2 Loss...

- **how** do we make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$?
- More generally: how do we measure the difference between two probability distributions?



Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

What this says:

- “Everywhere that p has probability density...”
- “...the difference between p and q should be small”
 - Difference in log probabilities (remember that $\log \left(\frac{a}{b} \right) = \log(a) - \log(b)$)
 - Note that this is not symmetric

More on KL Divergence: <https://jessicstringham.net/2018/12/27/KL-Divergence/>

The Final VAE Loss Function

We now have all the tools necessary to construct our loss function.

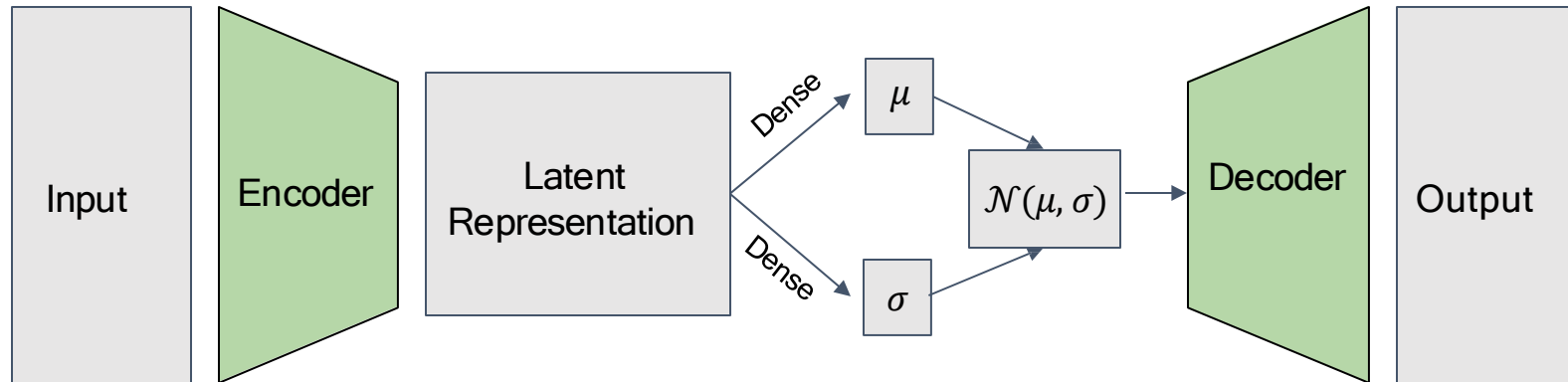
$$L = L_1 + \lambda L_2 \quad \lambda \in [0, \infty]$$

Which turns into this:

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$

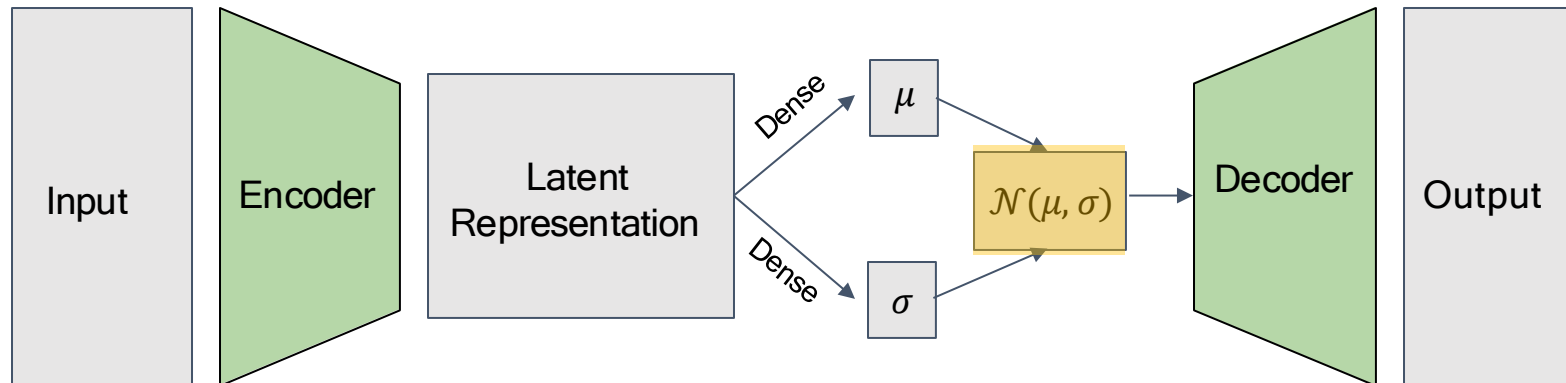
Putting it all together

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



Ah, but there's a catch:

- To update the weights of the encoder, we have to backprop through a random sampling operation
- Sampling a random value seems not differentiable...



The Reparameterization Trick

A nice property of Gaussian distributions: if we sample $z \sim \mathcal{N}(\mu, \sigma)$ we can rewrite it as:

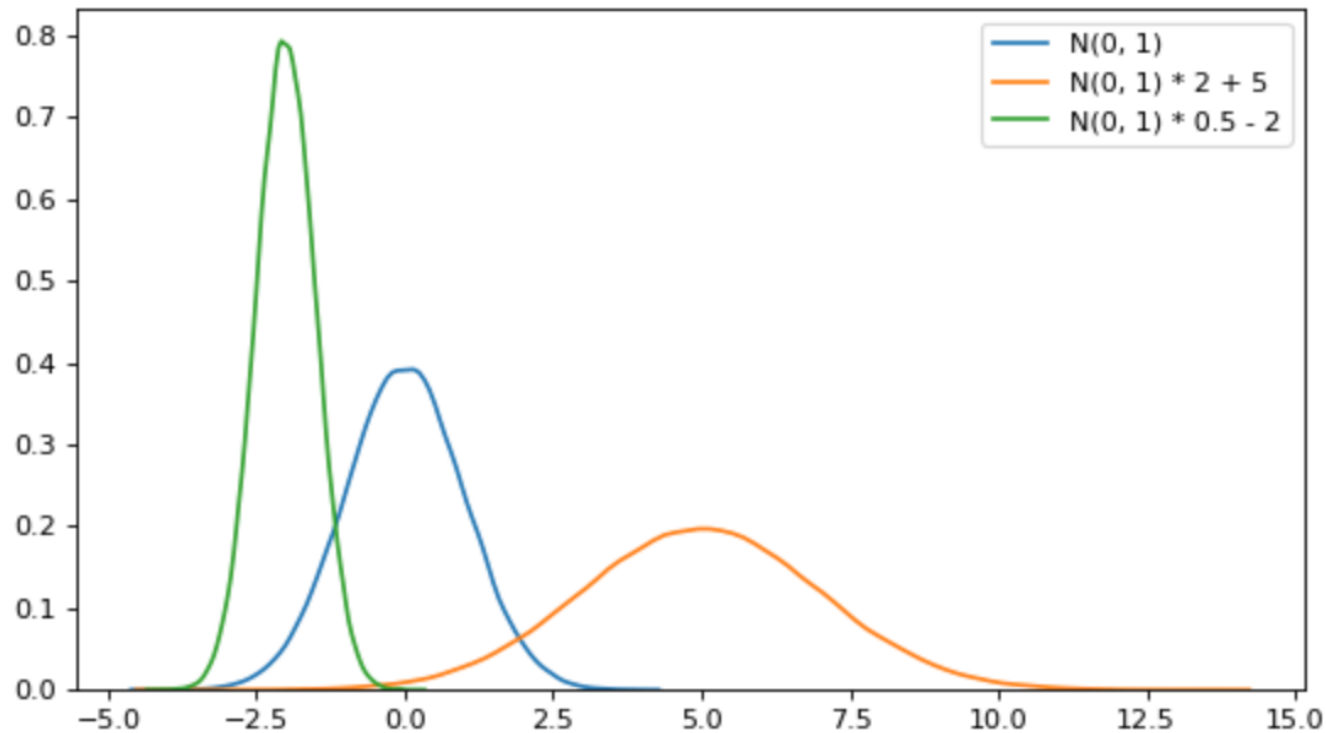
$$z = \mu + \epsilon \cdot \sigma$$

Where

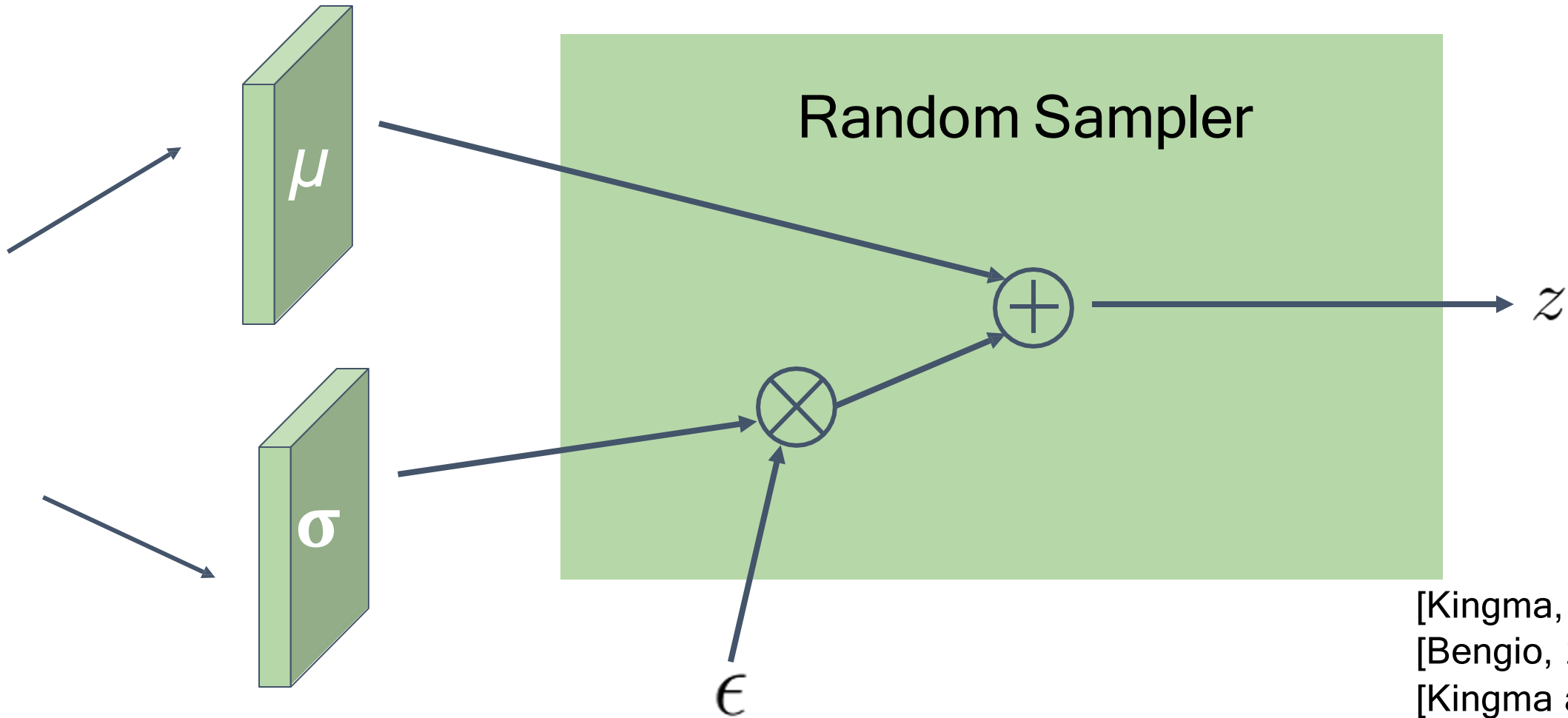
$$\epsilon \sim \mathcal{N}(0, 1)$$

Reparameterization Trick

Re-write $x \sim \mathcal{N}(x|\mu, \sigma^2)$ as $x = \mu + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(x|0, I)$

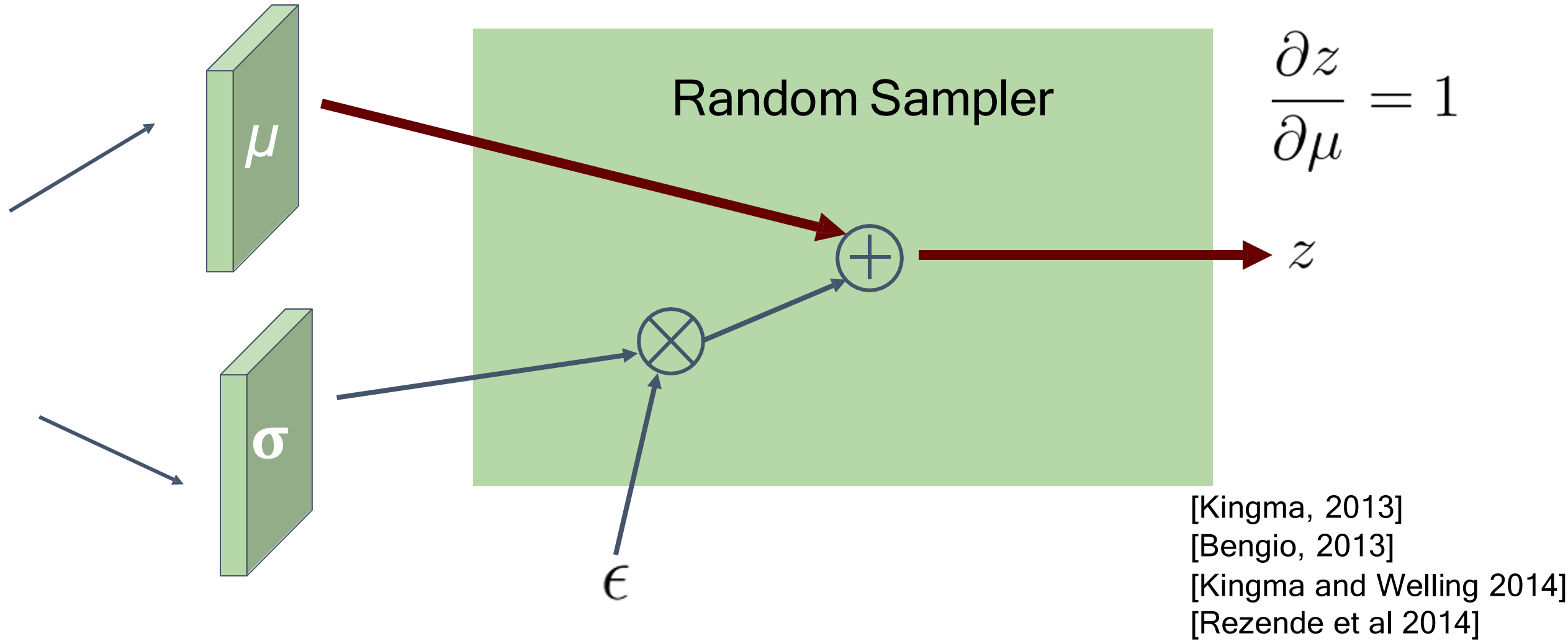


Random Sampler with Reparameterization Trick

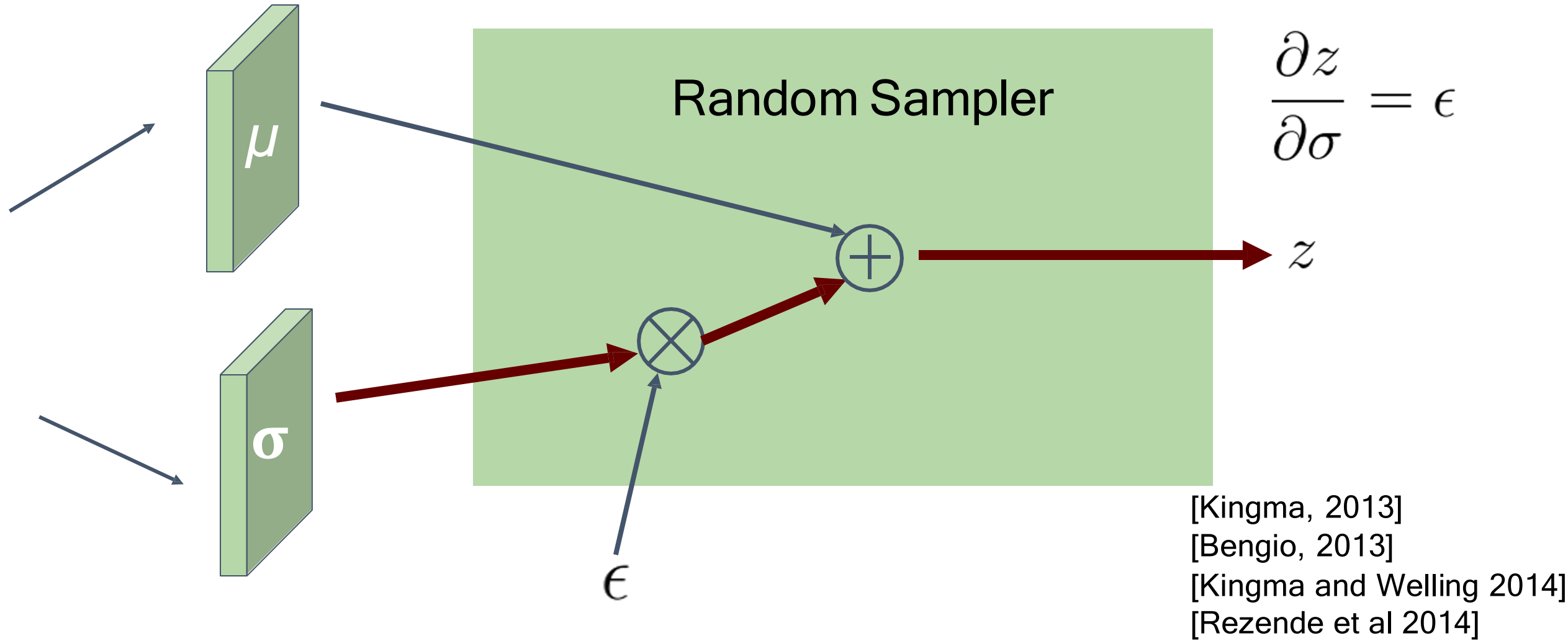


[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

Random Sampler with Reparameterization Trick

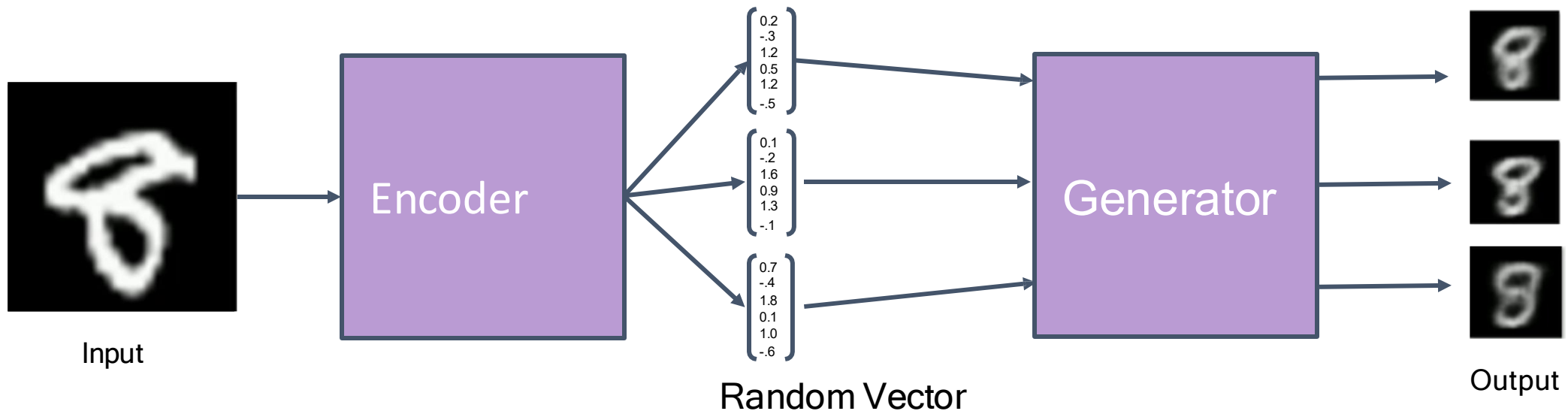


Random Sampler with Reparameterization Trick



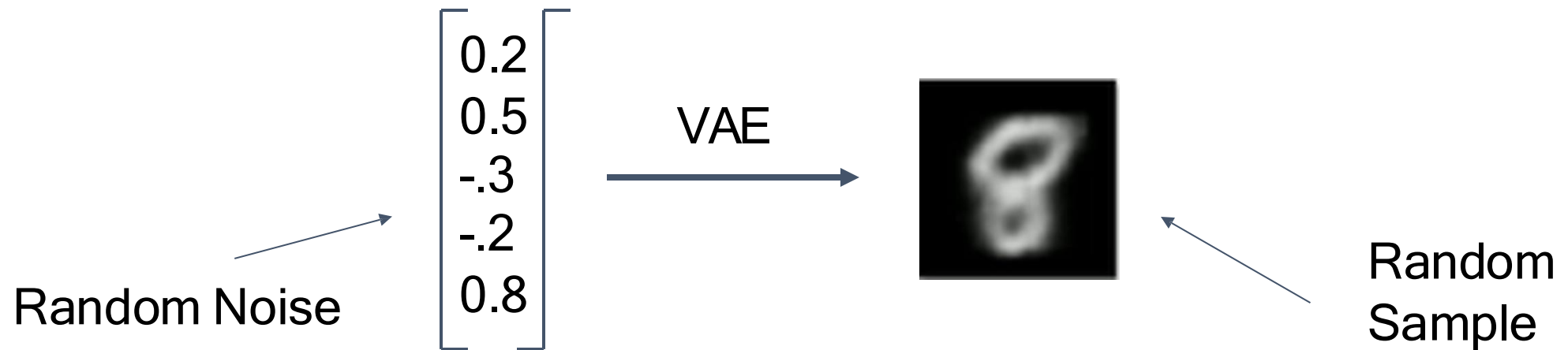
Sampling from a VAE

- We can use a trained VAE to generate random variants of an input data point



Sampling from a VAE

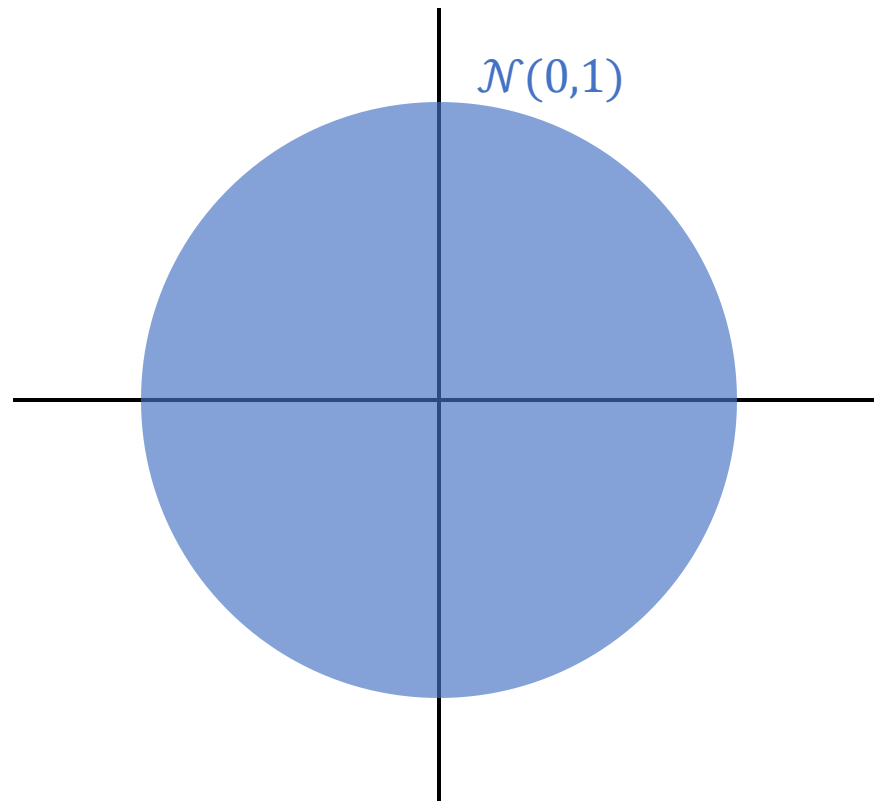
But ultimately, we want to draw random samples from a VAE



How can we do this?

This is where our particular choice of training loss will pay off

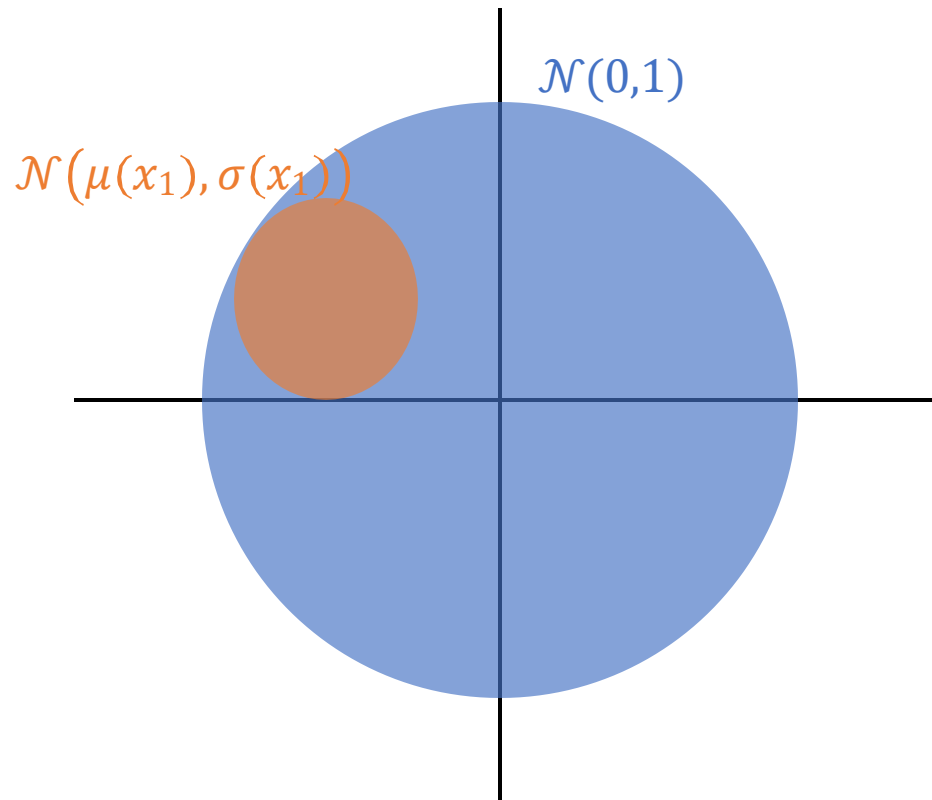
Encoding different points into latent space



Let this circle represent the probability density of a unit Gaussian in latent space

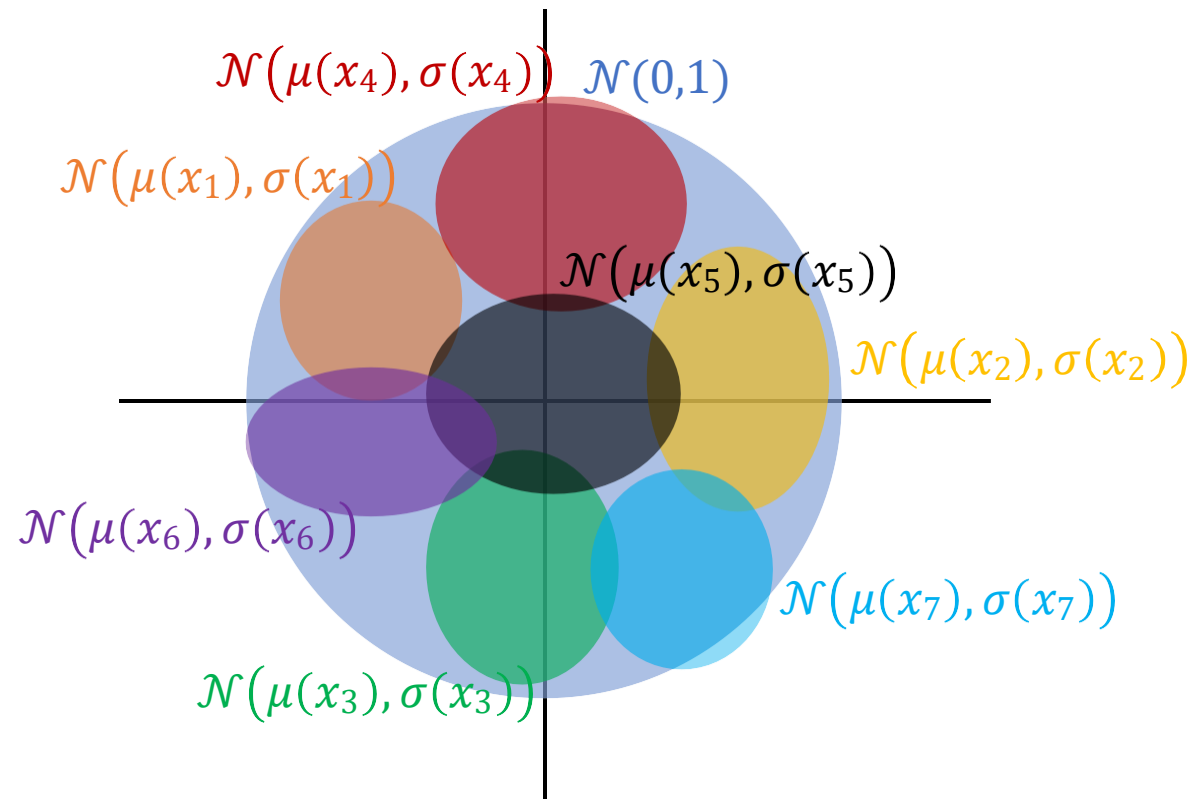
Encoding different points into latent space

Let this circle represent the probability density of the $\mathcal{N}(\mu, \sigma)$ distribution that the encoder predicts given an input data point x_1



Encoding different points into latent space

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$

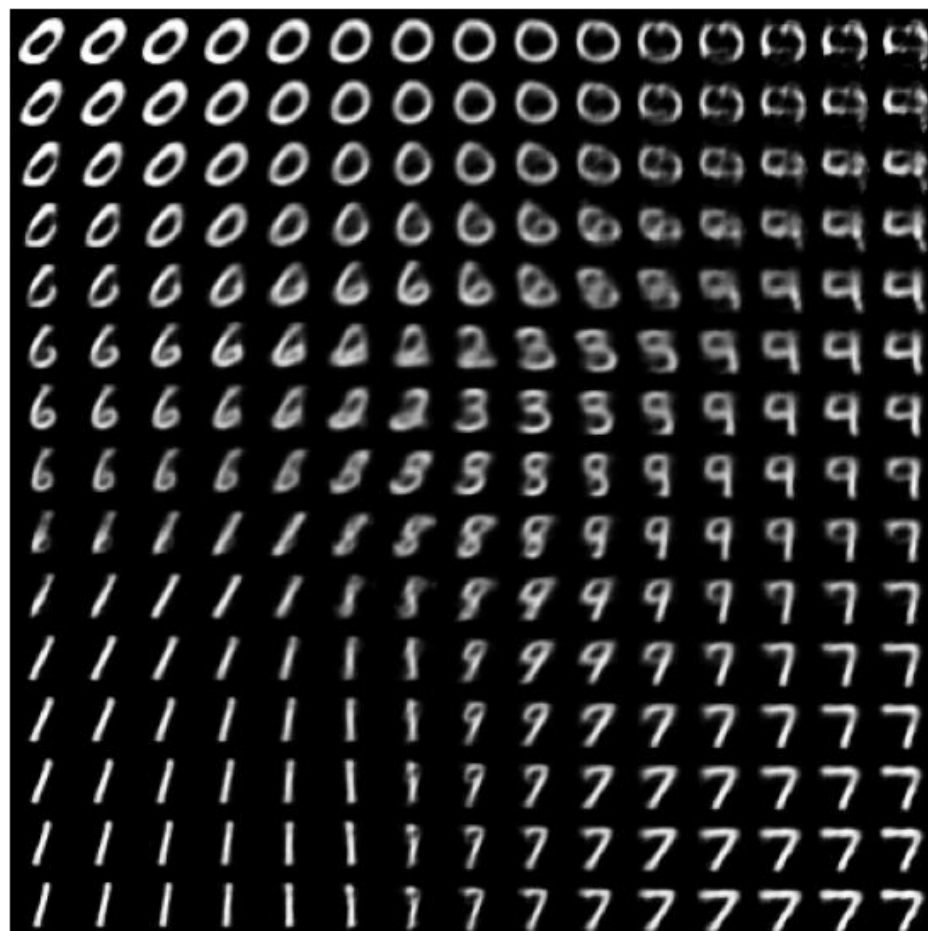


Because of our KL divergence loss, the $\mathcal{N}(\mu, \sigma)$ for any input data point has to be somewhat similar to $\mathcal{N}(0,1)$

So, if we sample a point from $\mathcal{N}(0,1)$, it is very likely to fall within one of these encoded distributions from the training set...

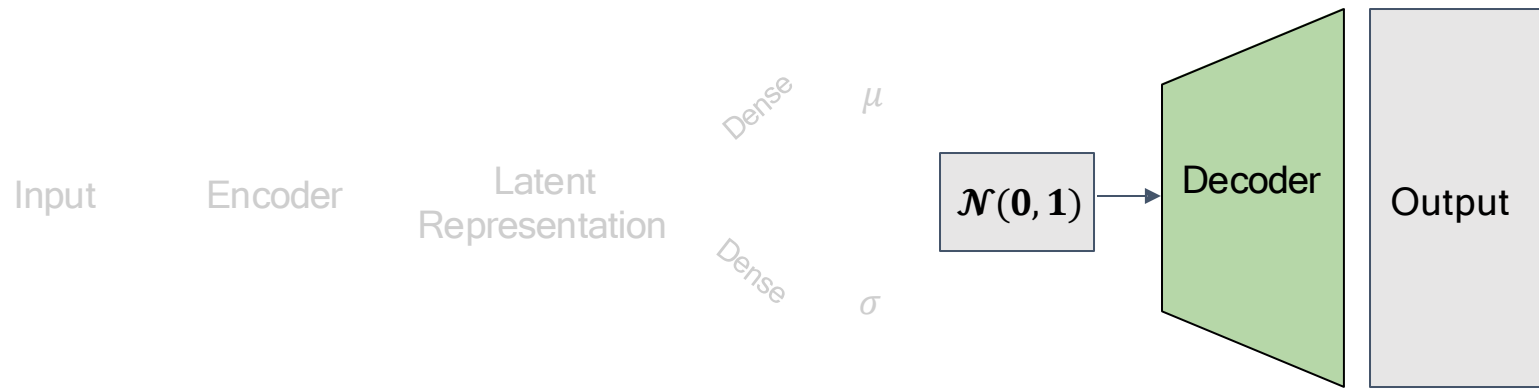
...which the decoder has been trained to reconstruct well!

VAE on MNIST



(trained by Alec Radford 2015)

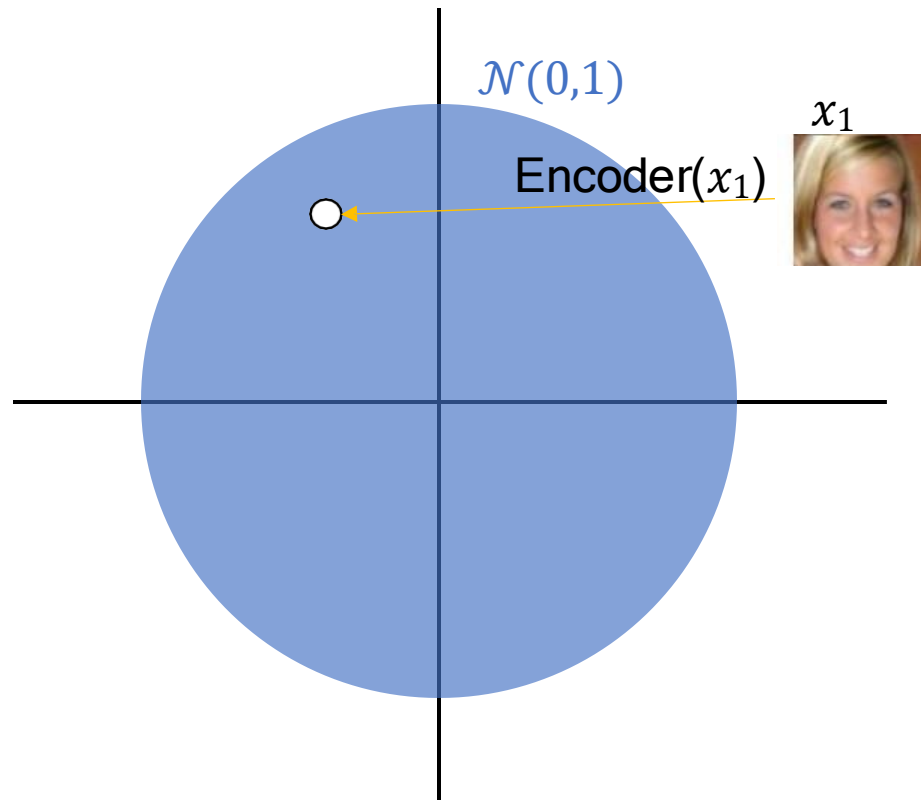
Sampling from a VAE



- Discard this part of the network...
- ...and set $(\mu, \sigma) = (0, 1)$

Latent Space Interpolation

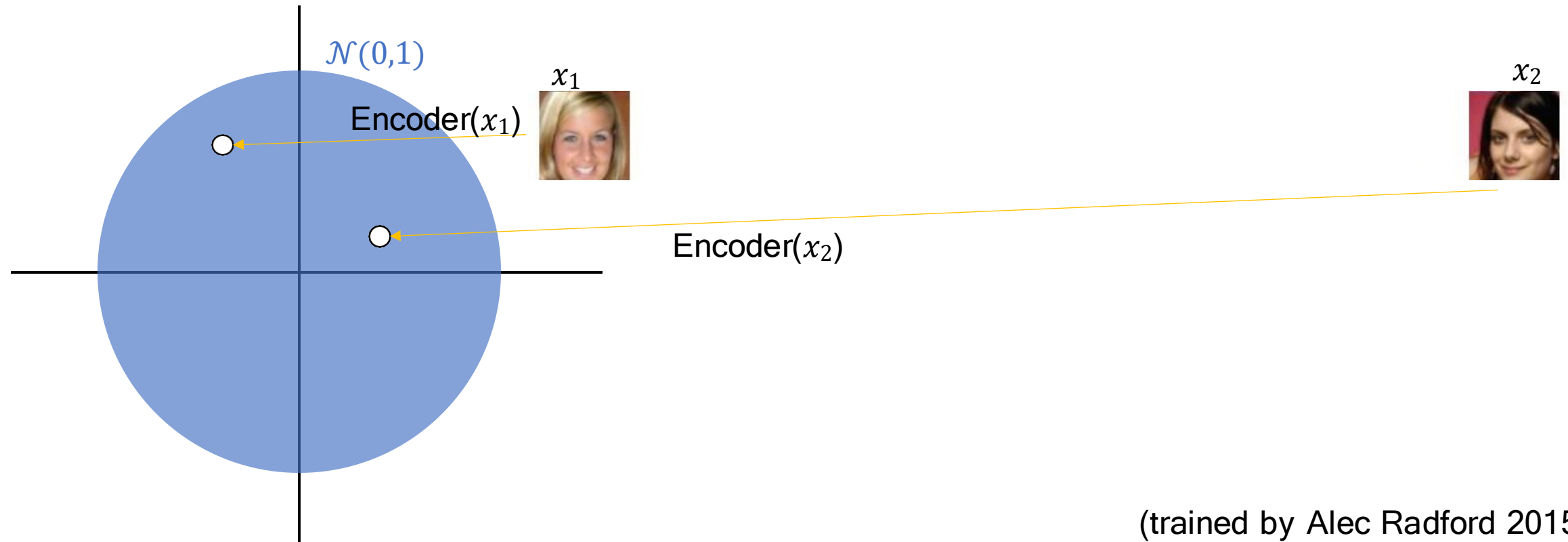
- Trace a linear path between two points in latent space, put all points along the path into the decoder



(trained by Alec Radford 2015)

Latent Space Interpolation

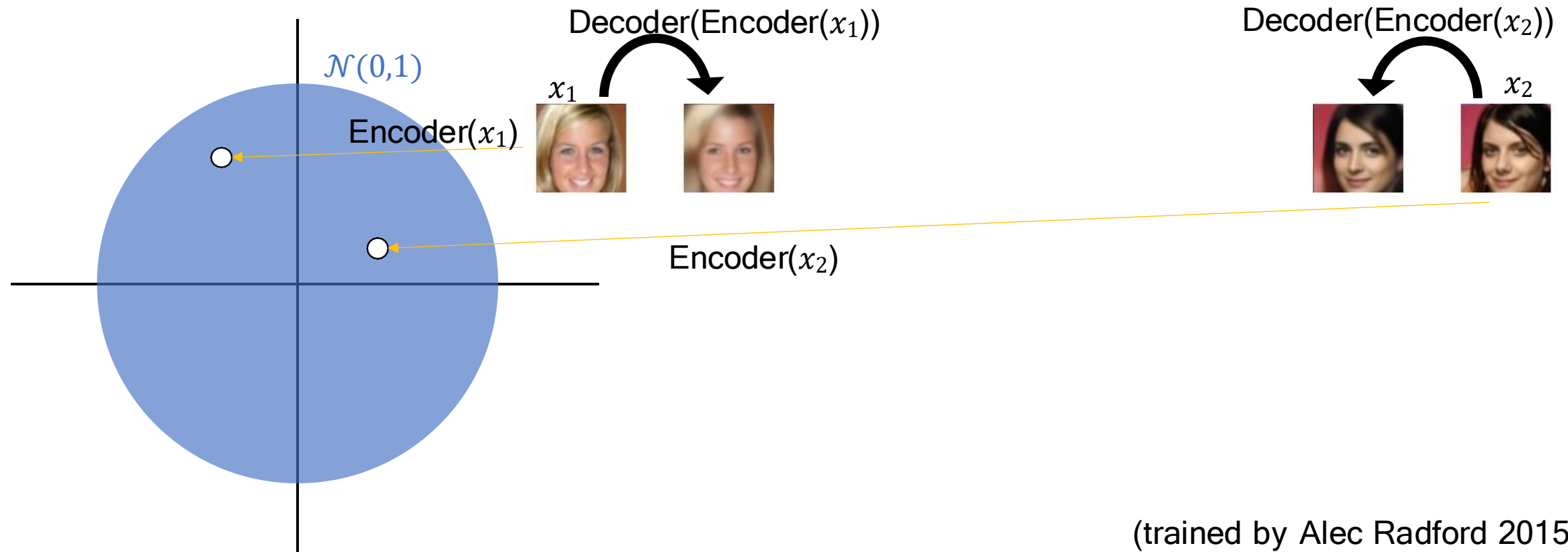
- Trace a linear path between two points in latent space, put all points along the path into the decoder



(trained by Alec Radford 2015)

Latent Space Interpolation

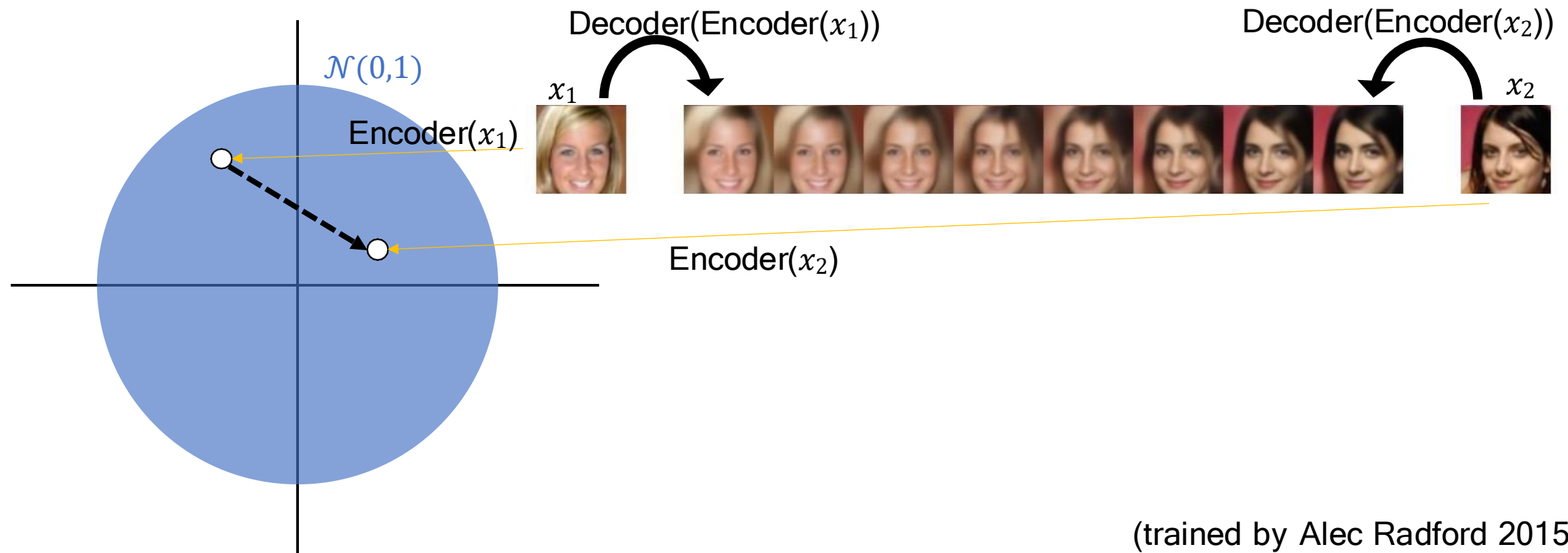
- Trace a linear path between two points in latent space, put all points along the path into the decoder



(trained by Alec Radford 2015)

Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder

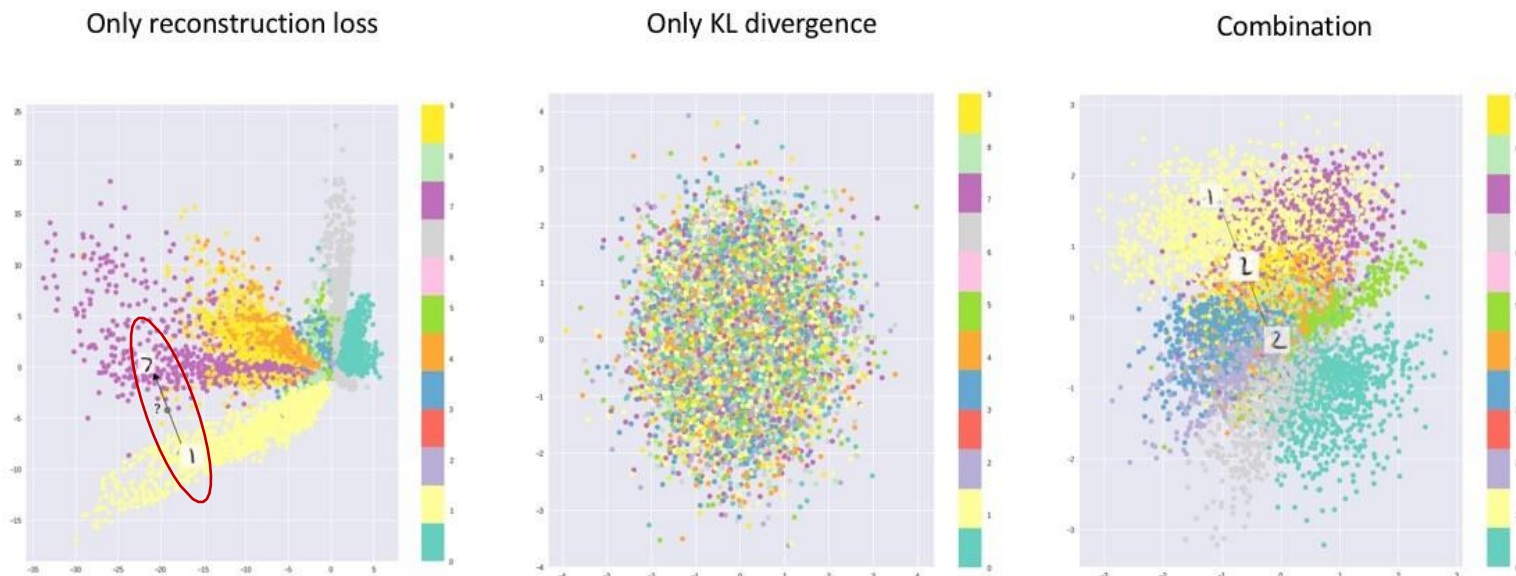


(trained by Alec Radford 2015)

Latent Space Interpolation

- Can also try it with a regular autoencoder
 - Doesn't work as well
 - ***Why not?***
 - The KL divergence loss regularizes the shape of the latent space. Without it, a regular autoencoder might have “empty” pockets of latent space

Linear interpolation has to cross a pocket of empty space (where the behavior of the decoder is not well defined)



Discriminative vs Generative Models

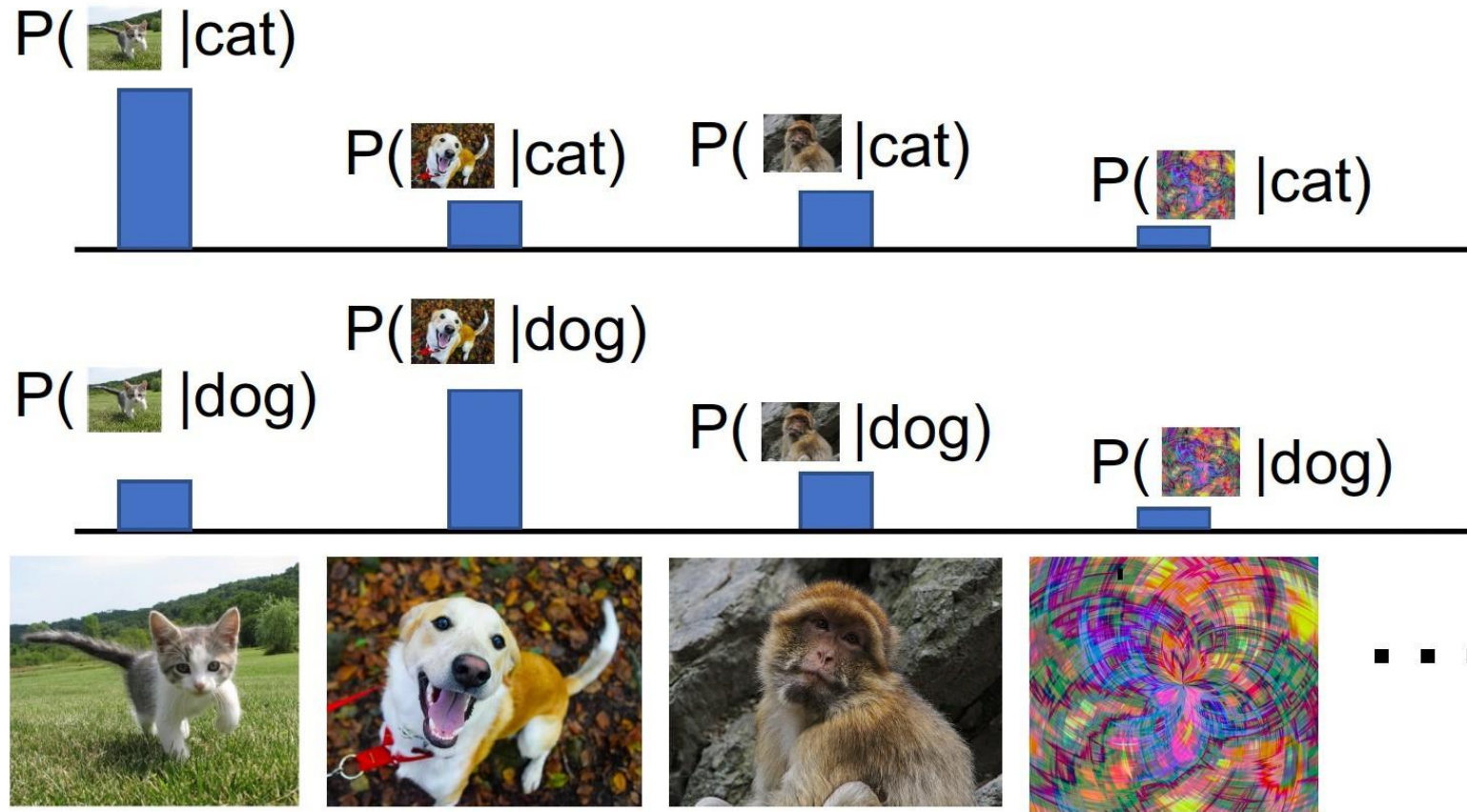
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

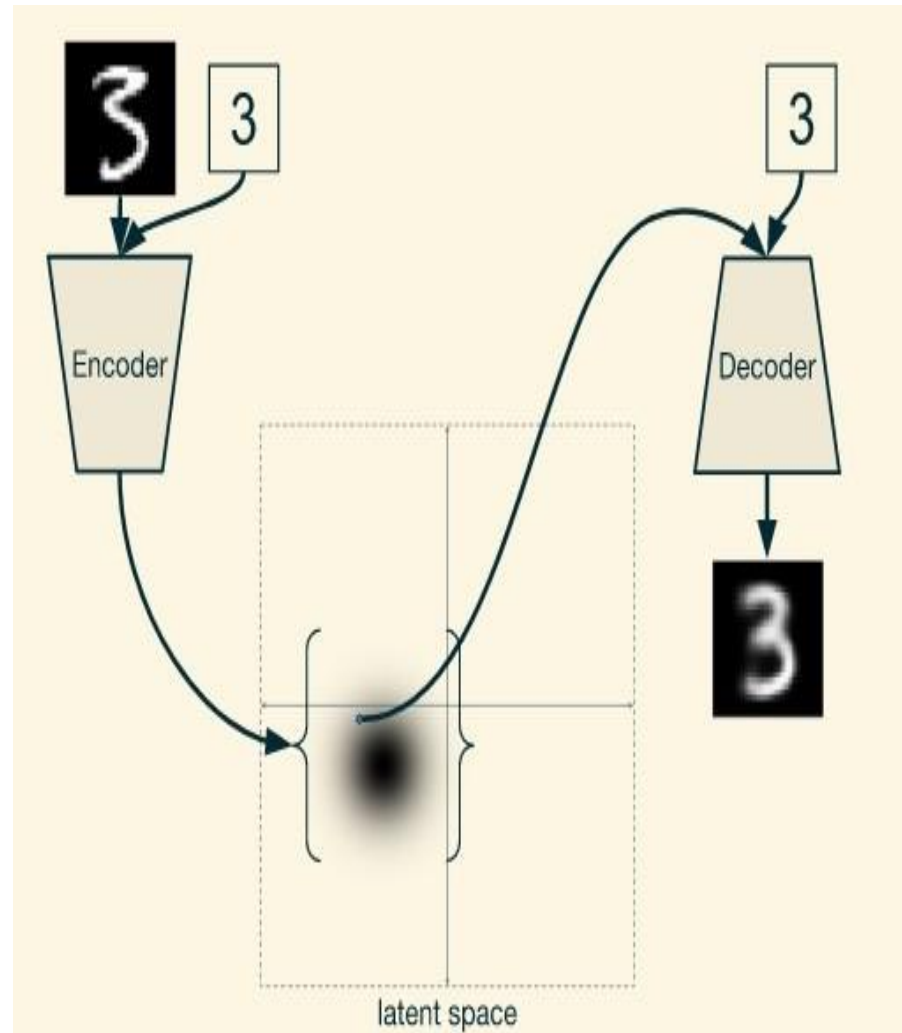
Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$



Conditional Generative Model: Each possible label induces a competition among all images

Conditional VAE



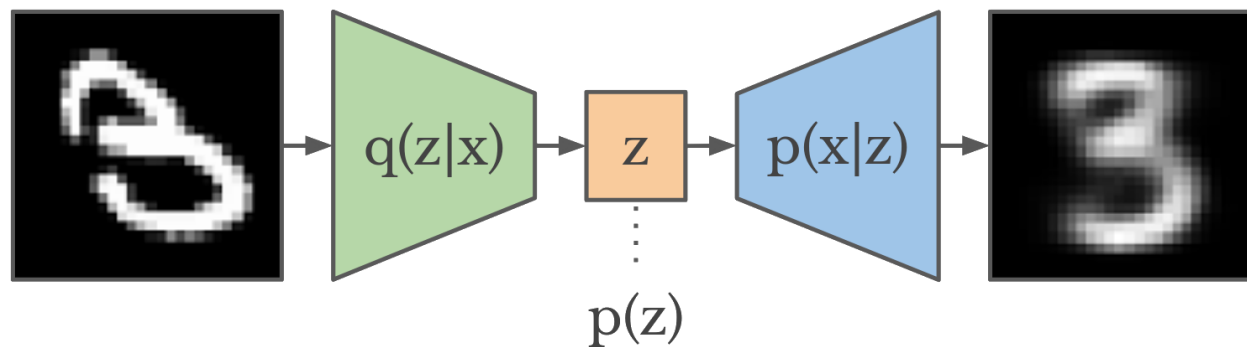
Next, diffusion process - intuition

The idea is **to systematically and slowly destroy structure** in a data distribution through an iterative forward diffusion process.
Then **to learn a reverse diffusion process that restores structure in data**, yielding a highly flexible and tractable generative model of the data.

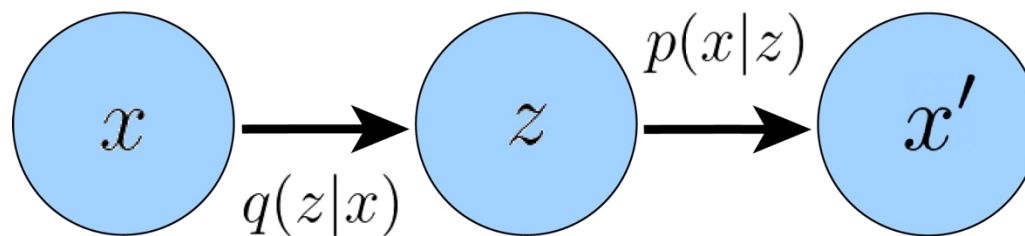
“Destroy structure” and learn the reverse process



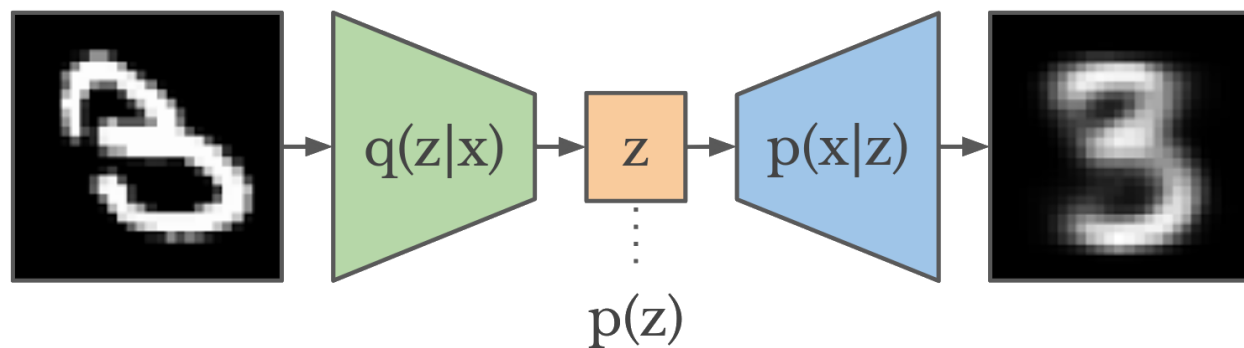
VAEs notations



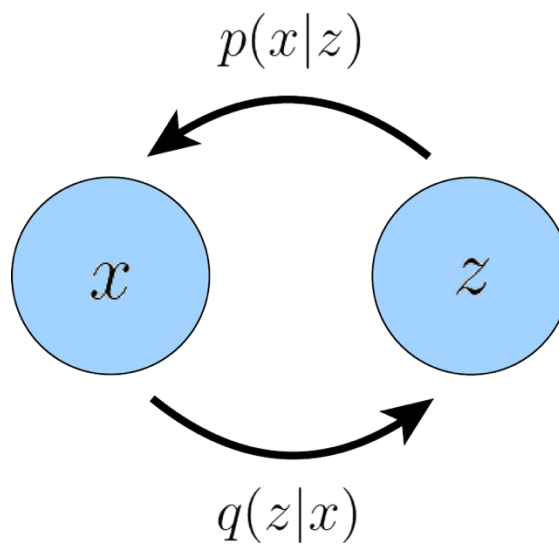
Or as



VAEs notations

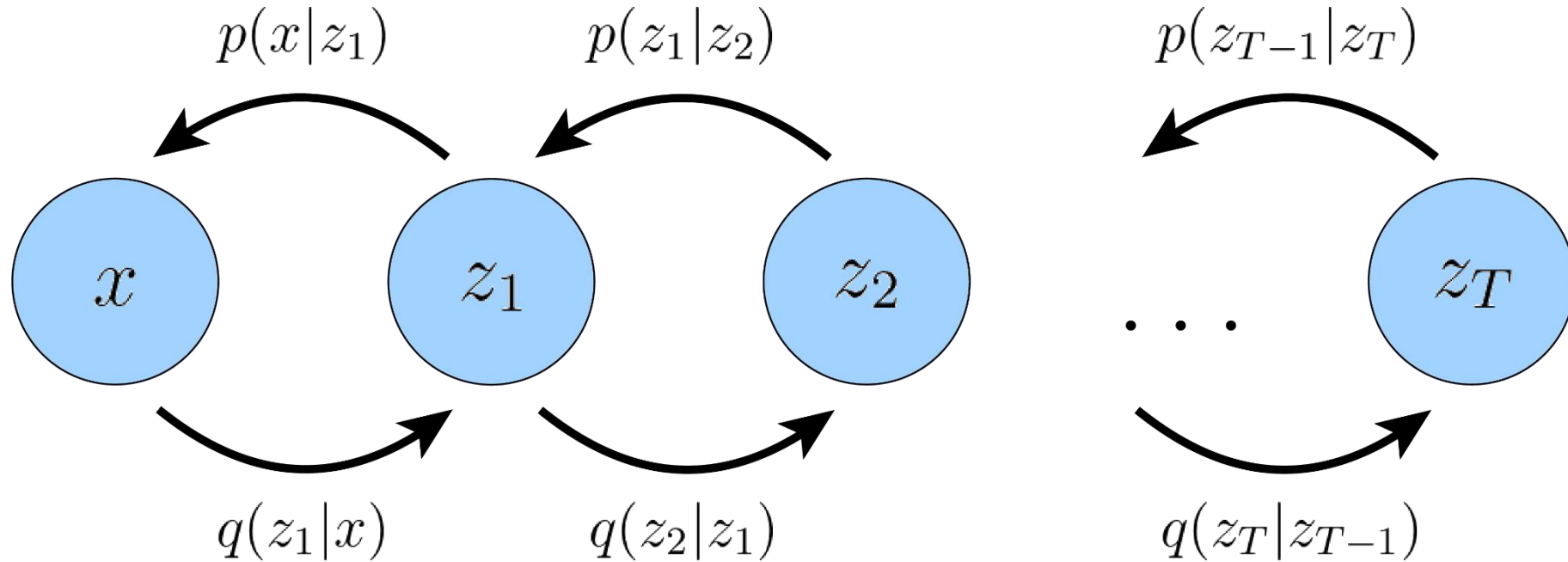


Or as



Diffusion can be seen as Hierarchical VAEs

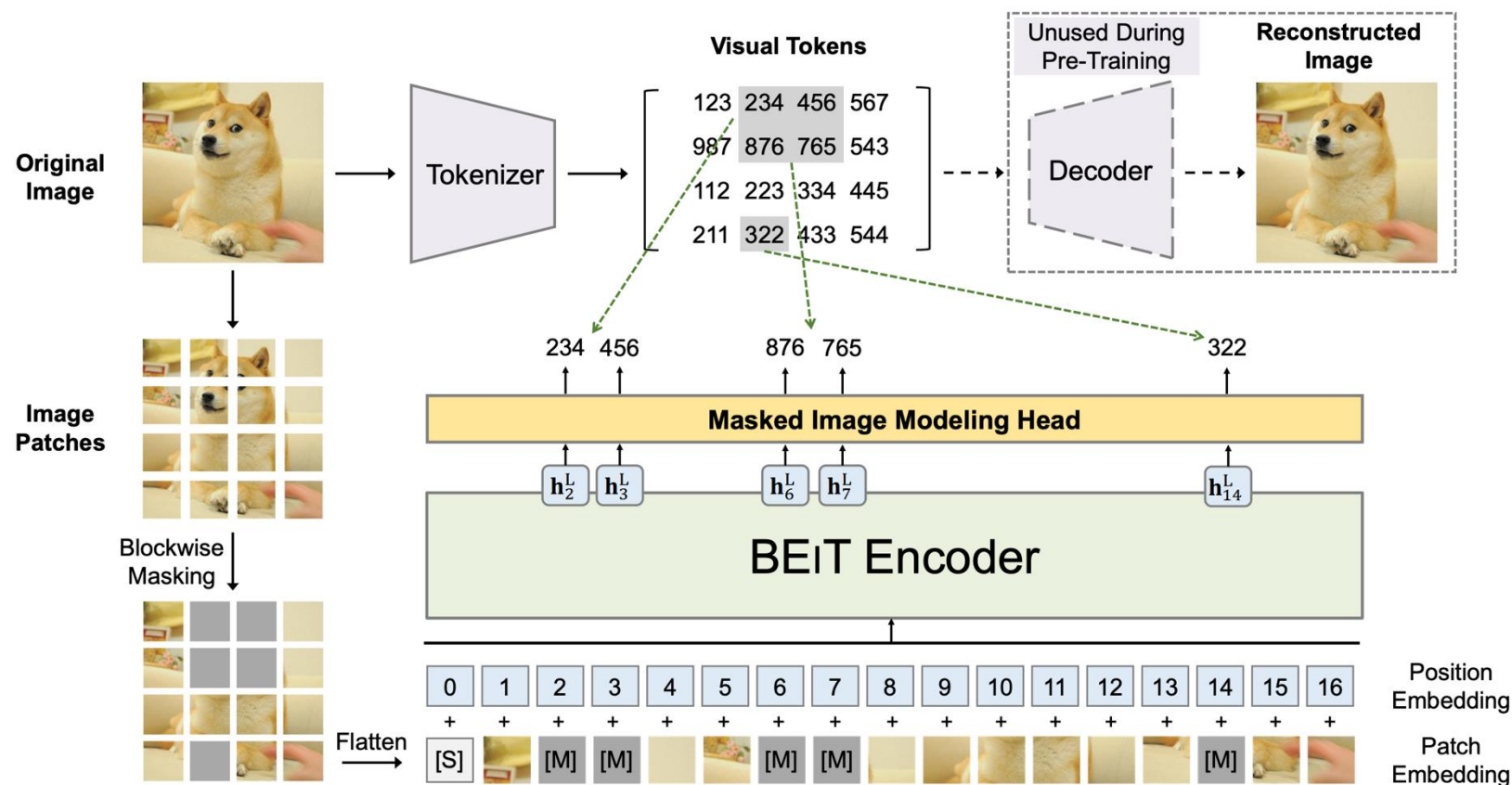
- Generalize VAEs by enabling a hierarchy of latents $z_1, \dots, z_T$



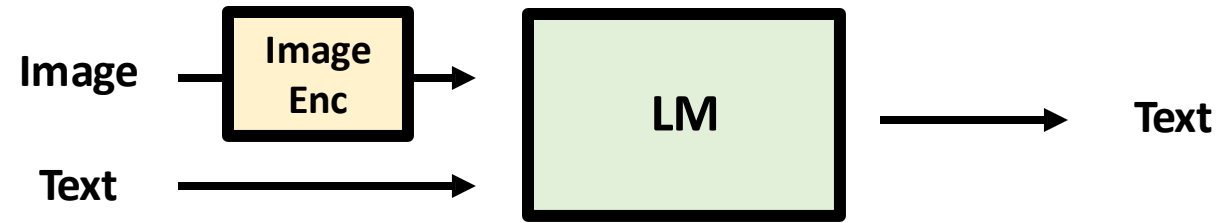
More on the next lecture

Appendix

Multimodal Self-supervised: Visual Tokens

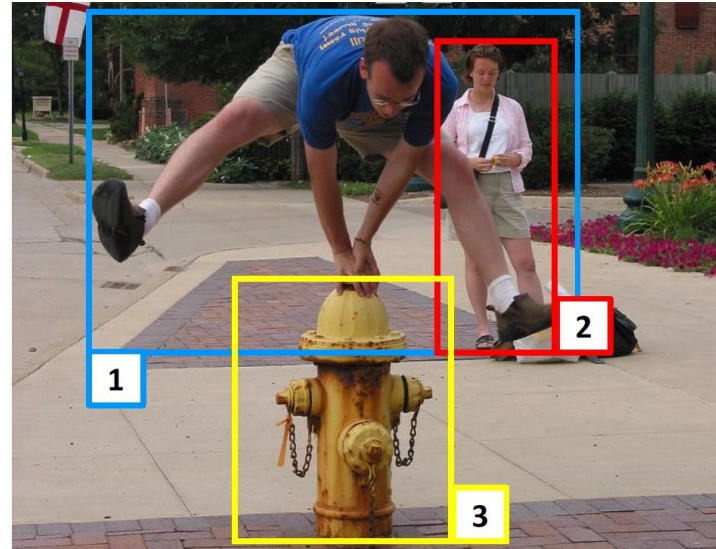


Can we tackle all V&L tasks with a single objective?



Vision-Language T5

Many Multimodal Tasks as Text Generation



Text Input



Text Output

Multimodal
LM

“span prediction: A <text_1> is
<text_2> over fire hydrant”



“<text_1> man
<text_2> jumping”

Visual QA

“vqa: what is the man jumping over?”



“fire hydrant”

Visual
Grounding

“visual grounding: yellow fire hydrant”



“<vis_3>”

Image-Text
Matching

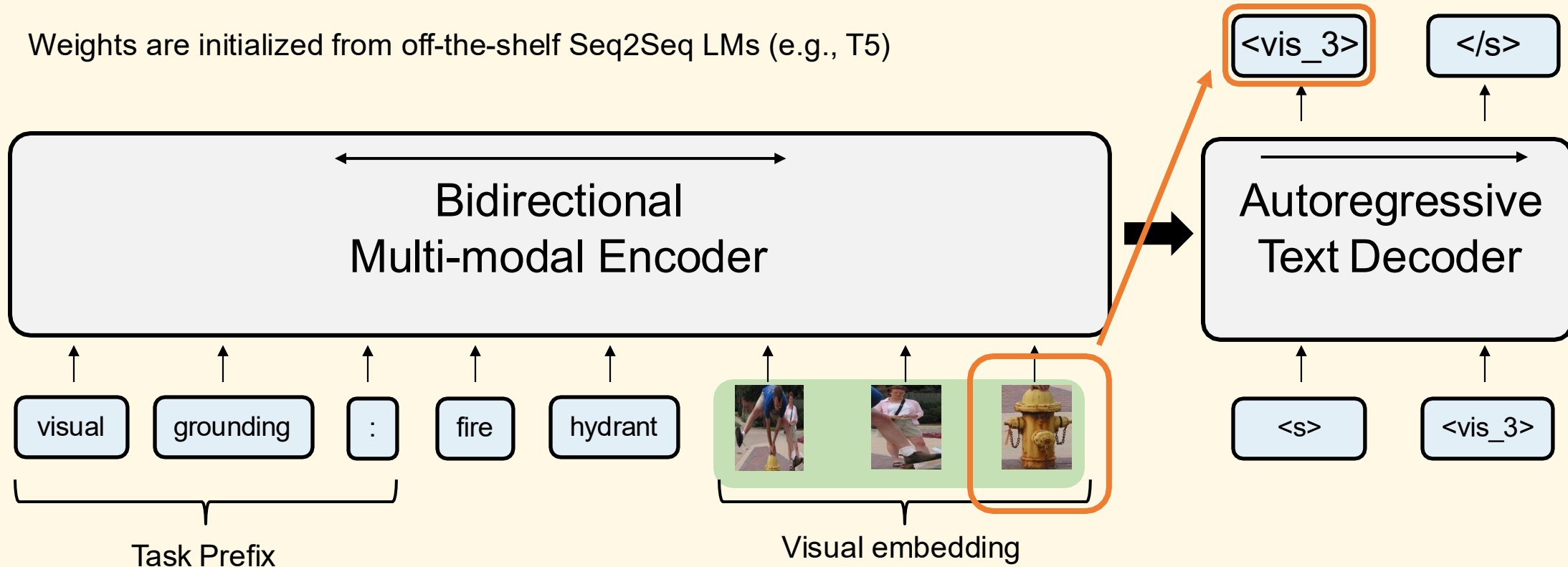
“image text match: A cat is lying on a
bed”



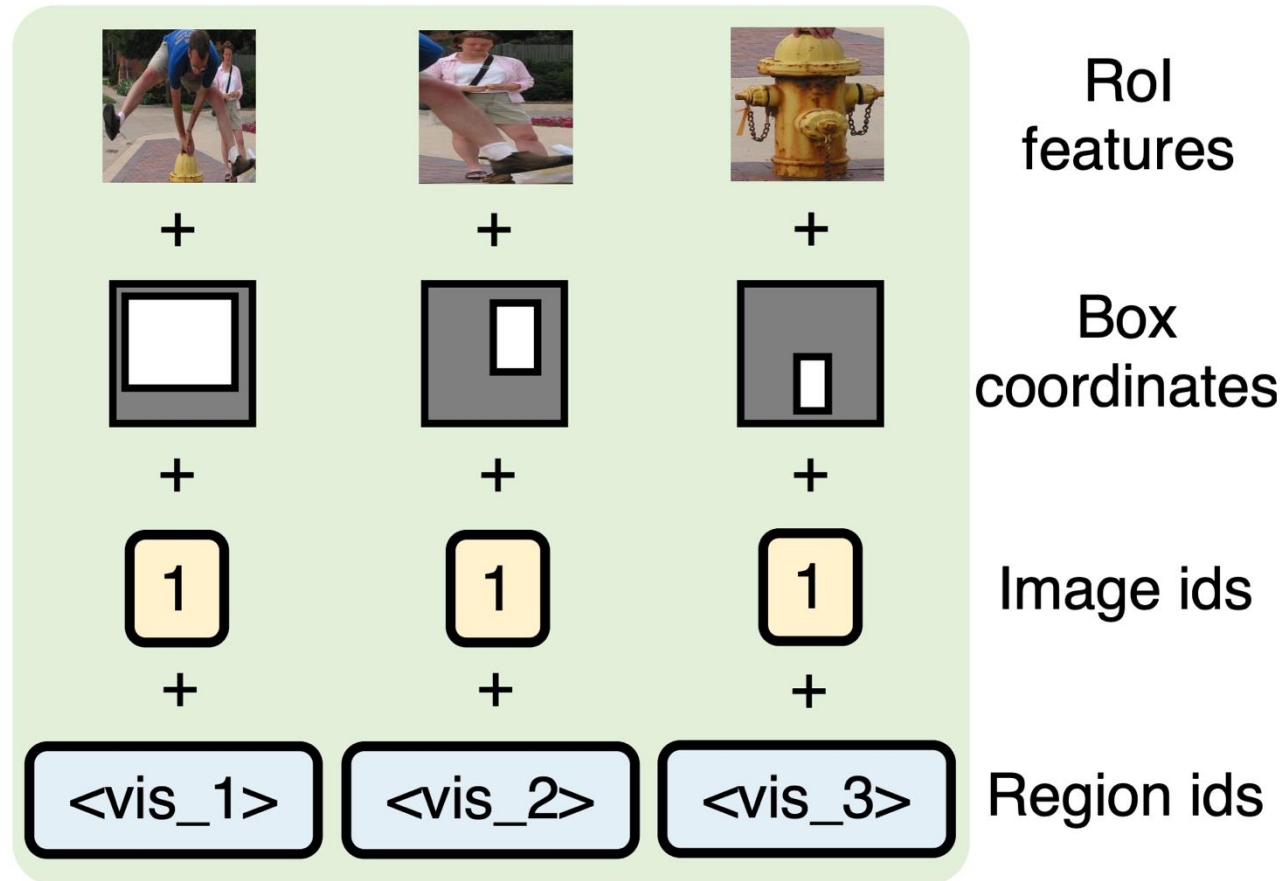
“false”

VL-T5 Architecture

Weights are initialized from off-the-shelf Seq2Seq LMs (e.g., T5)

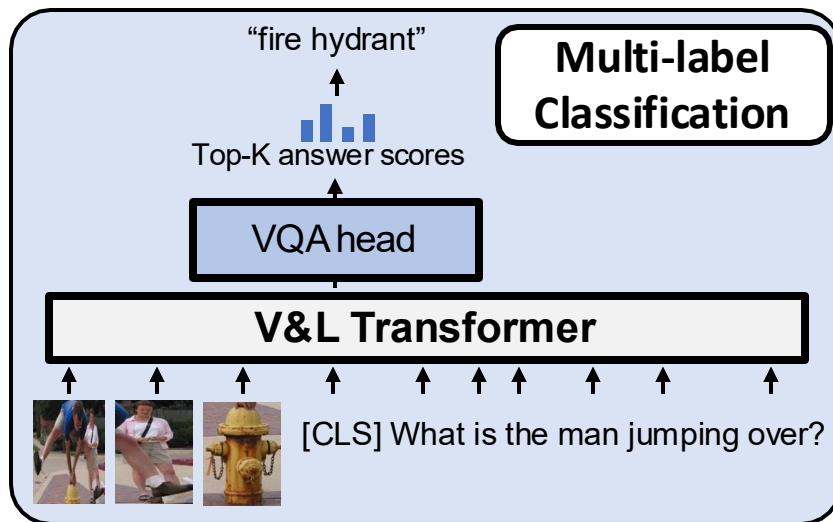


Visual embeddings

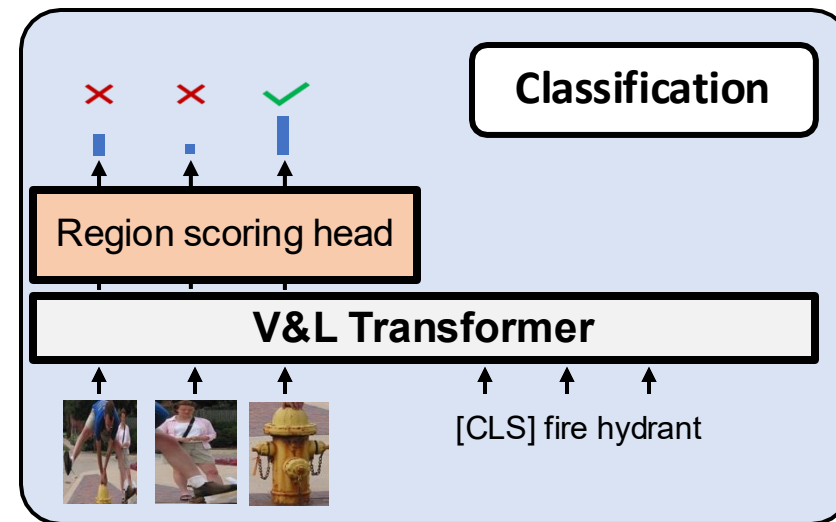


Previous models

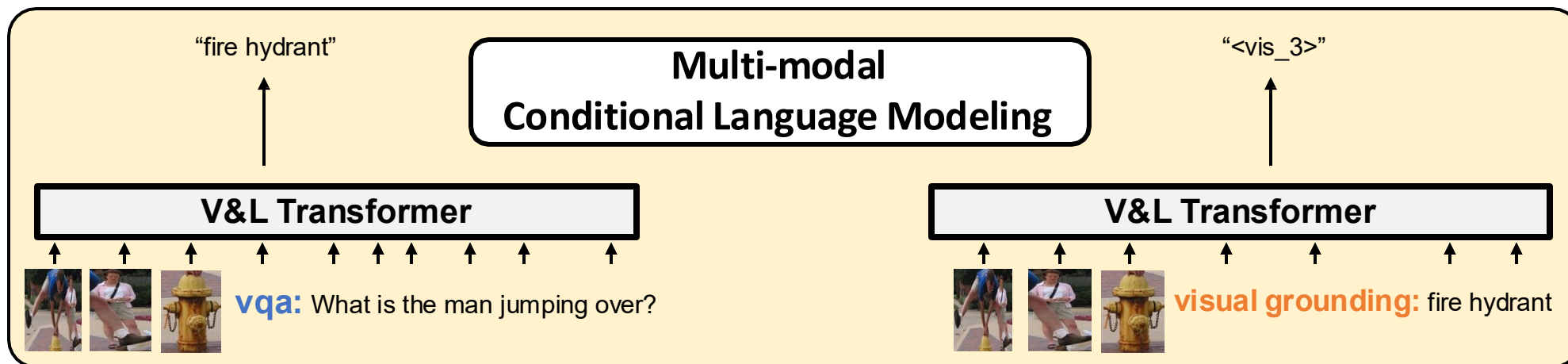
Visual Question Answering



Visual Grounding



VL-T5



Next-latent prediction maybe a promising direction for SSL?

Next-Embedding Prediction Makes Strong Vision Learners

Sihan Xu¹ Ziqiao Ma¹ Wenhao Chai³ Xuweiyi Chen⁴ Weiyang Jin^{†2}
Joyce Chai¹ Saining Xie² Stella X. Yu¹

predicted to predict unknown

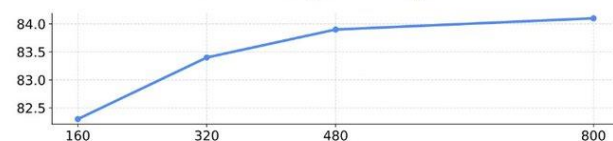


embed

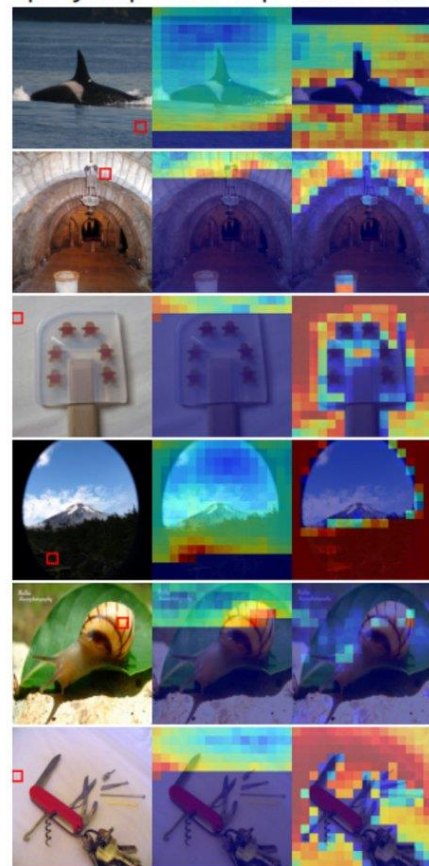
input



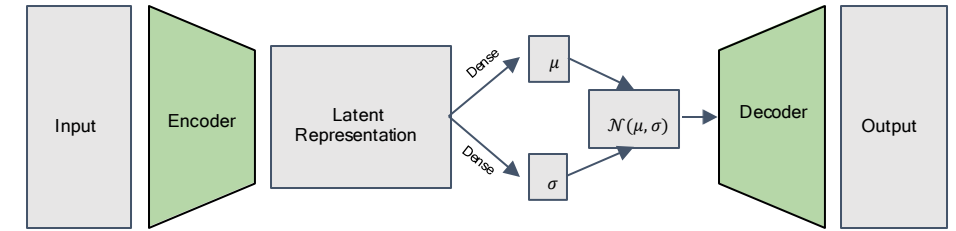
NEPA-L ImageNet Top-1 Acc



query map attn. map cosine sim.



One more practical detail



Let's again consider our sampling operation

$$z \sim \mathcal{N}(\mu, \sigma)$$

$\mu_i \in [-\infty, \infty]$ $\sigma_i \in [0, \infty]$

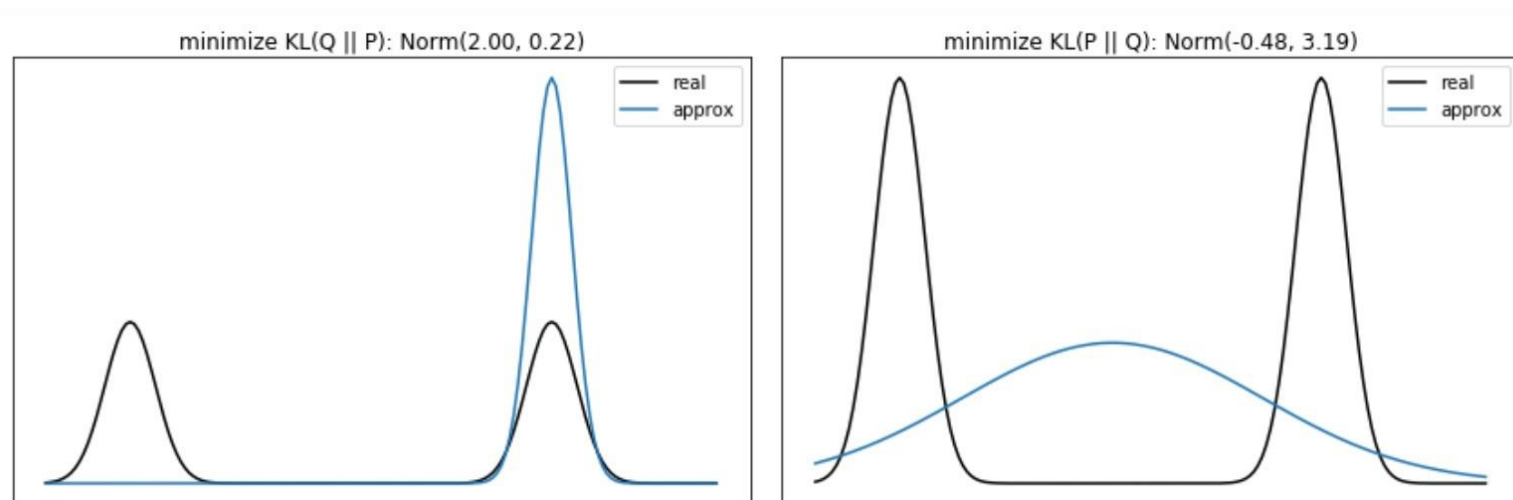
- Nothing prevents the neural network from outputting ***negative*** values for the standard deviation.
- Instead of predicting σ , we will instead predict $\log(\sigma^2)$. This ensures that every $\sigma_i \in [0, \infty]$
 - i.e. just treat the output of the Dense layer as if it is $\log(\sigma^2)$

Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

- Note that this is not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$



Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

k is the dimensionality of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (e.g. $k = 100$ when $\mu \in \mathbb{R}^{100}$)

We won't derive the equation above, but let's convince ourselves it behaves how we expect it to behave

Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Derive the expression for (1) $\sigma = 1$ and (2) $\mu=0$

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\sigma = 1$

$$\begin{aligned} D_{KL}(\mathcal{N}(\mu, 1) || \mathcal{N}(0, 1)) &= \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + 1^2 - \ln(1) - 1) \\ &= \frac{1}{2} \sum_{i=1}^k \mu_i^2 \end{aligned}$$

The expression is minimized $\mu = 0$ (which is what we want)

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\mu = 0$

$$D_{KL}(\mathcal{N}(0, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 - \ln(\sigma_i^2) - 1)$$

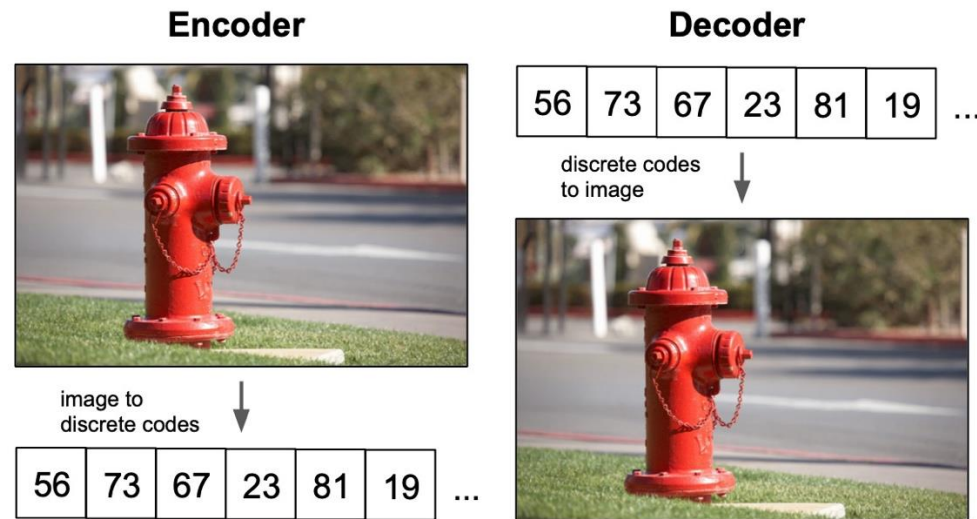
This expression is minimized when $\sigma = 1$ (which is also what we want)

Comparable to specialized models

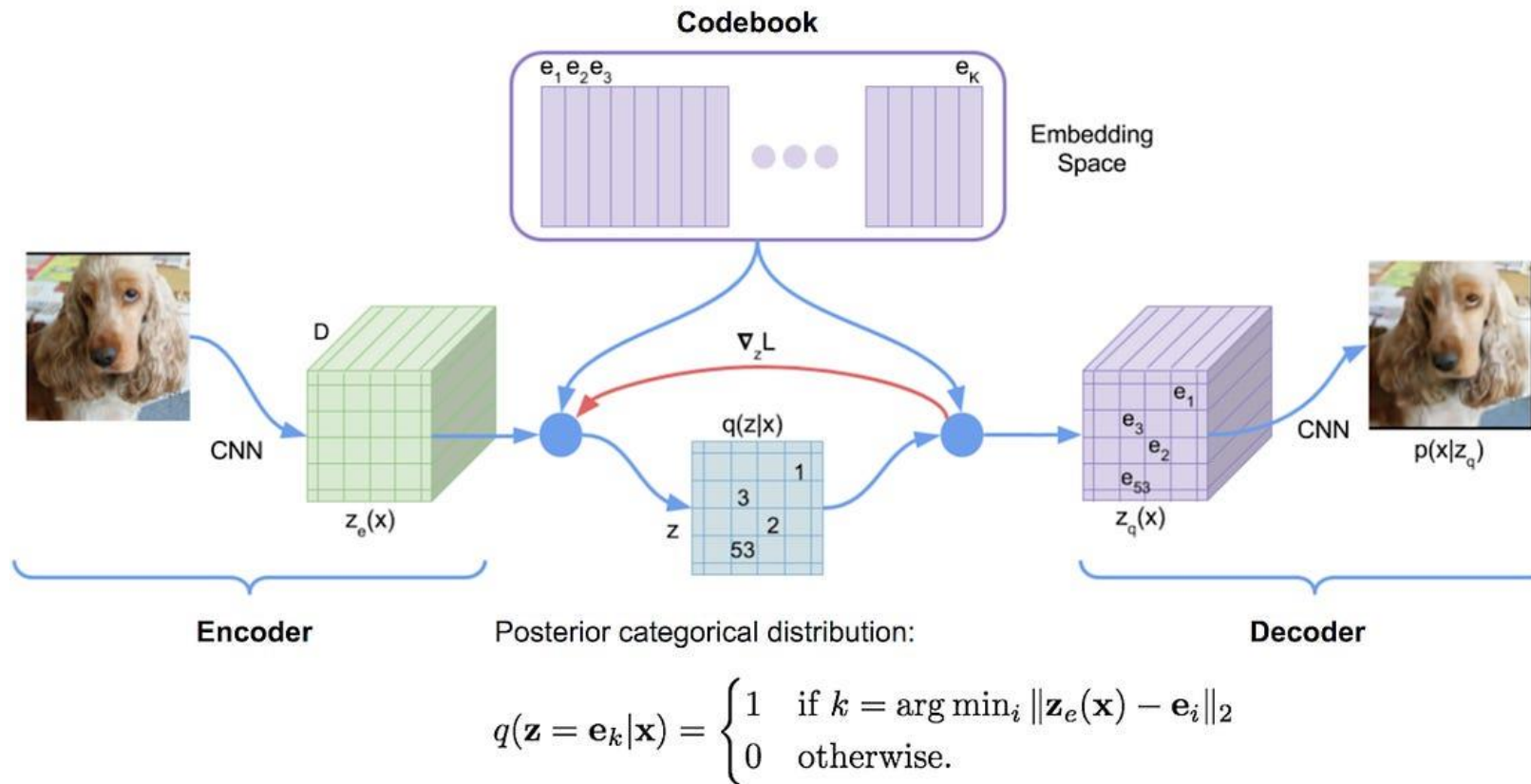
Method	# Pretrain Images	Discriminative tasks					Generative tasks	
		VQA	GQA	NLVR ²	RefCOCOg	VCR Q→AR	COCO Cap	Multi30K En-De
		test-std Acc	test-std Acc	test-P Acc	test ^d Acc	test Acc	Karpathy test CIDEr	test 2018 BLEU
LXMERT	180K	72.5	60.3	74.5	-	-	-	-
ViLBERT	3M	70.9	-	-	-	54.8	-	-
UNITER _{Base}	4M	72.9	-	77.9	74.5	58.2	-	-
Unified VLP	3M	70.7	-	-	-	-	117.7	-
Oscar _{Base}	4M	73.4	61.6	78.4	-	-	123.7	-
XGPT	3M	-	-	-	-	-	120.1	-
MeMAD	-	-	-	-	-	-	-	38.5
VL-T5	180K	70.3	60.8	73.6	71.3	58.9	116.5	38.6
VL-BART	180K	71.3	60.5	70.3	22.4 [*]	48.9	116.6	28.1

Vector Quantized Variational Autoencoder (VQ-VAE)

- So far, we have seen how continuous vector spaces can be used to represent the latents in an autoencoder.
- But latents do not necessarily need to be continuous vectors



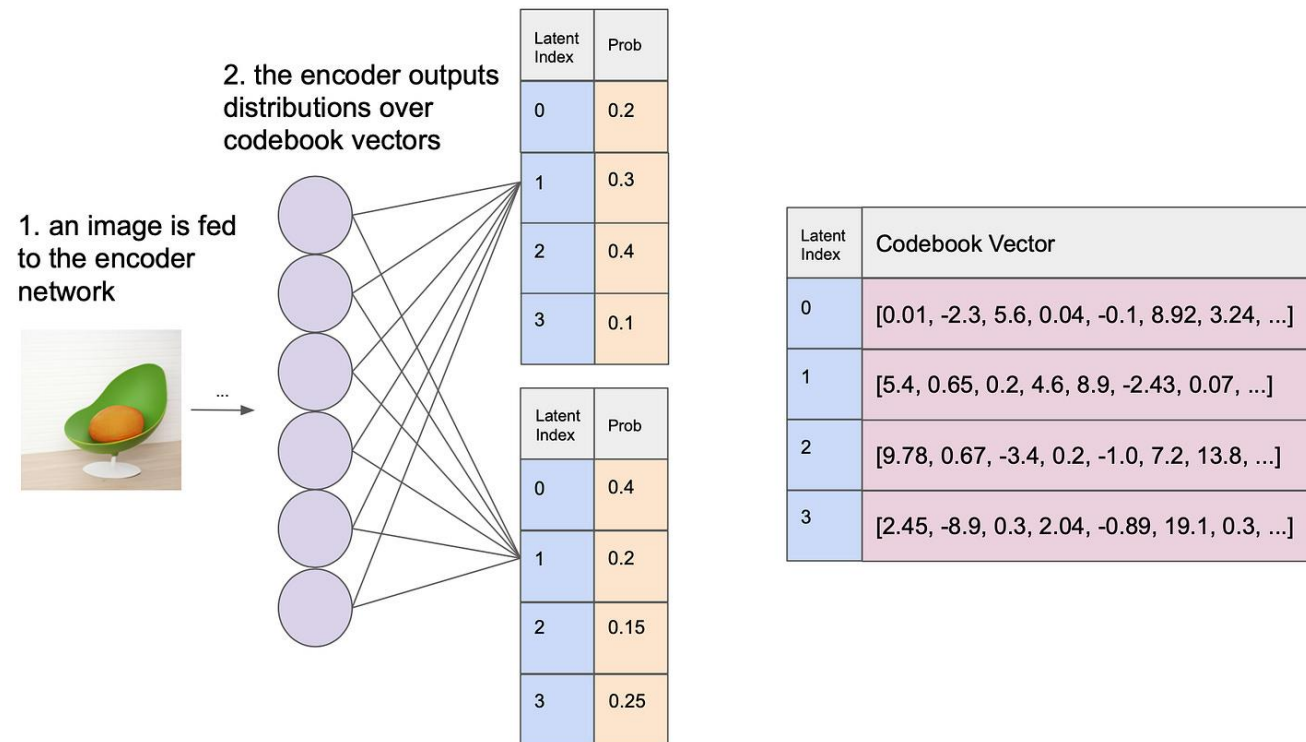
VQ-VAE training



Pick a Codebook index by taking the closest codebook vector
The min operation require tricks to backprop

dVAE

- Instead, we can use a soft approach that predicts probability for each latent[†]



But given the probability, how can we sample for each latent?

The Gumbel Softmax Relaxation

- We use weighted sum

$$z = \sum_{j=1..k} y_j e_j$$

- Where

$$y_j = \text{softmax}(p), p_i \propto \frac{q(e_i|x) + g_i}{\tau}$$

- The random sampling comes by adding Gumbel distribution g_i (re-parametrization trick)

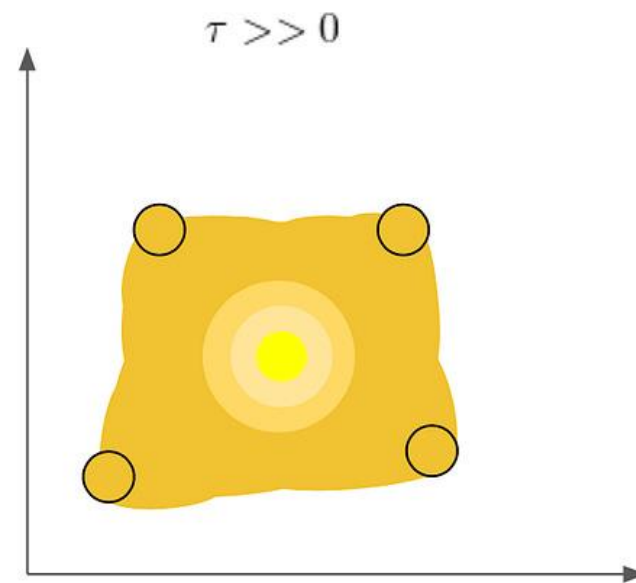
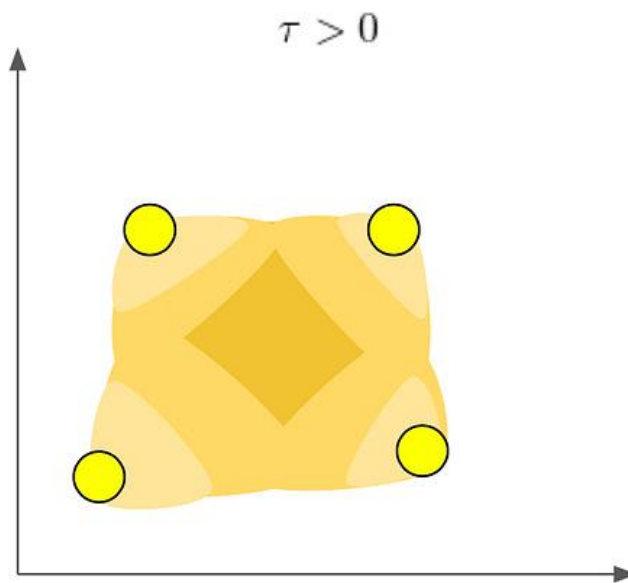
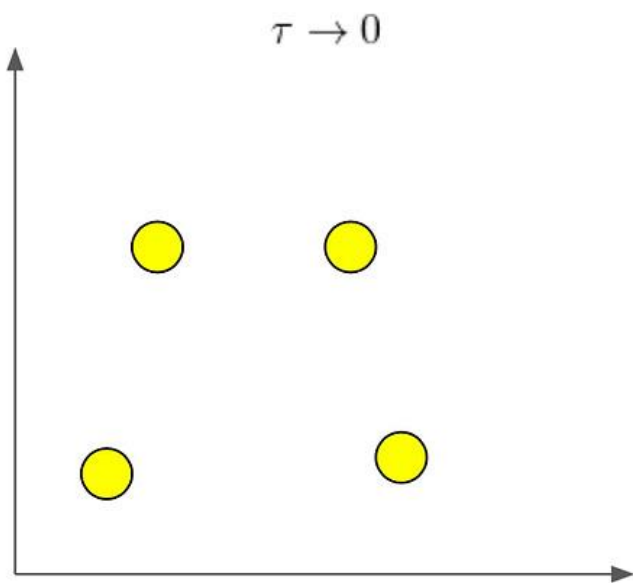
The temperature hyper-parameter

Gumbel Softmax distribution over latents for different ranges of τ

key:

 = codebook vector

brighter colors = higher probability



What about the KL term?

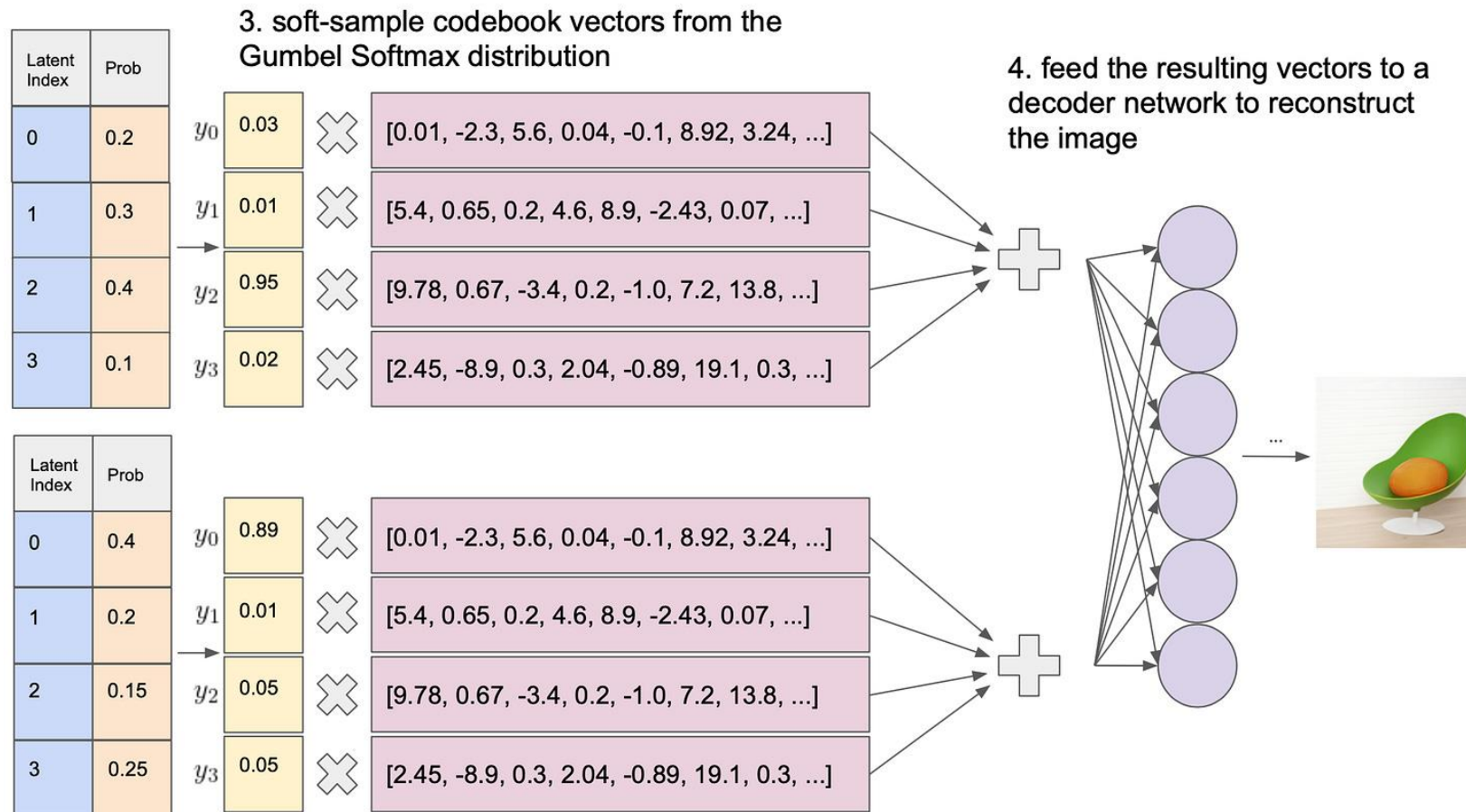
- In DALL-E's dVAE, the prior distribution ($p(z)$) is a uniform distribution over the discrete latent codes.
- This means that each latent code has an equal probability:

$$p(z) = \frac{1}{k}$$

- The close form is

$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^K q(z_i|x) \log q(z_i|x) + \log K$$

Encoding with dVAE



DALL-E

- DALL-E is one of the first successful
- It has 12 billion parameters, trained on 250 image-text pairs
- The main purpose is modeling $p(x|c)$; where c is a text prompt

TEXT PROMPT

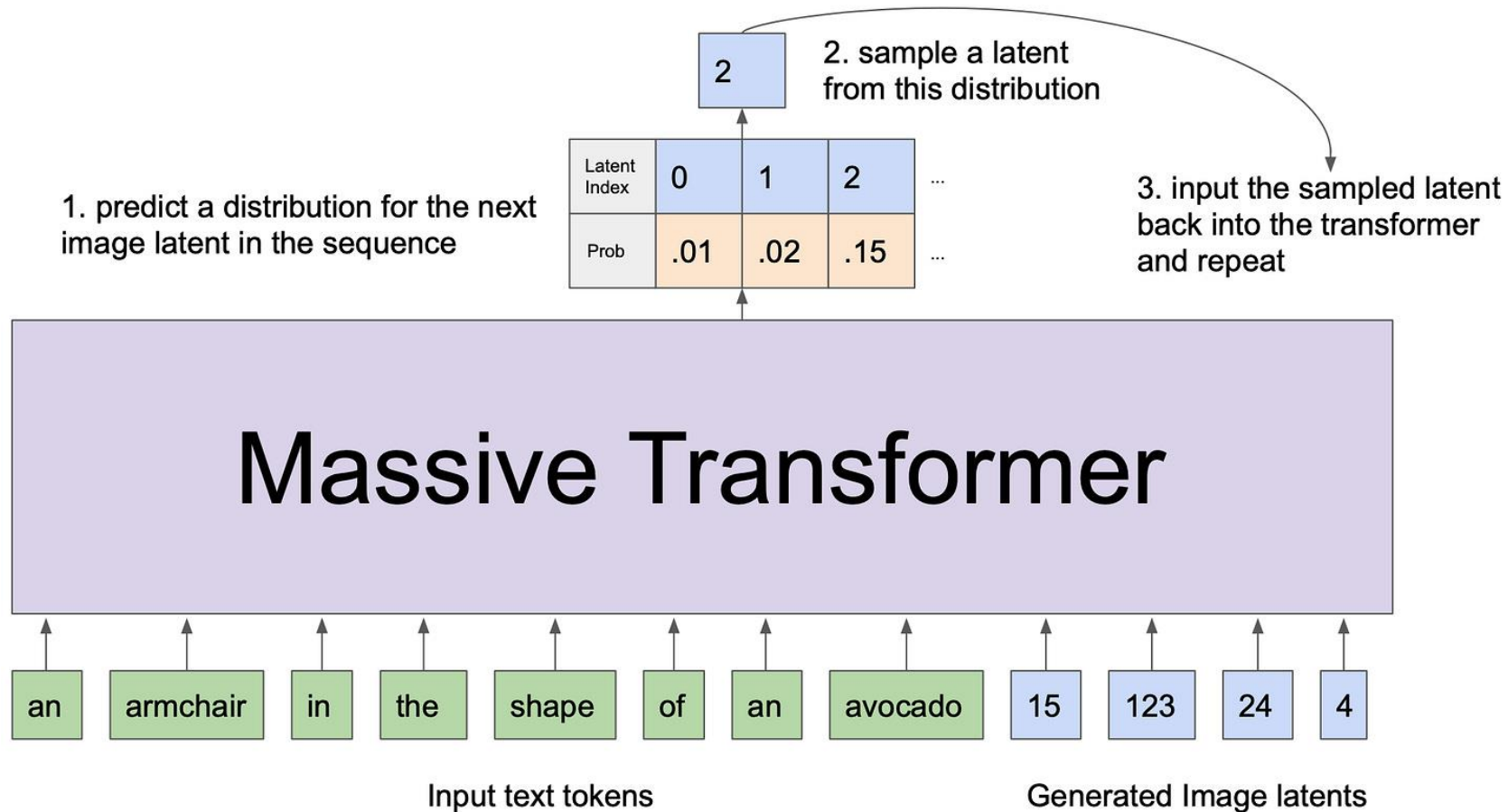
an armchair in the shape of an avocado [...]

AI-GENERATED IMAGES

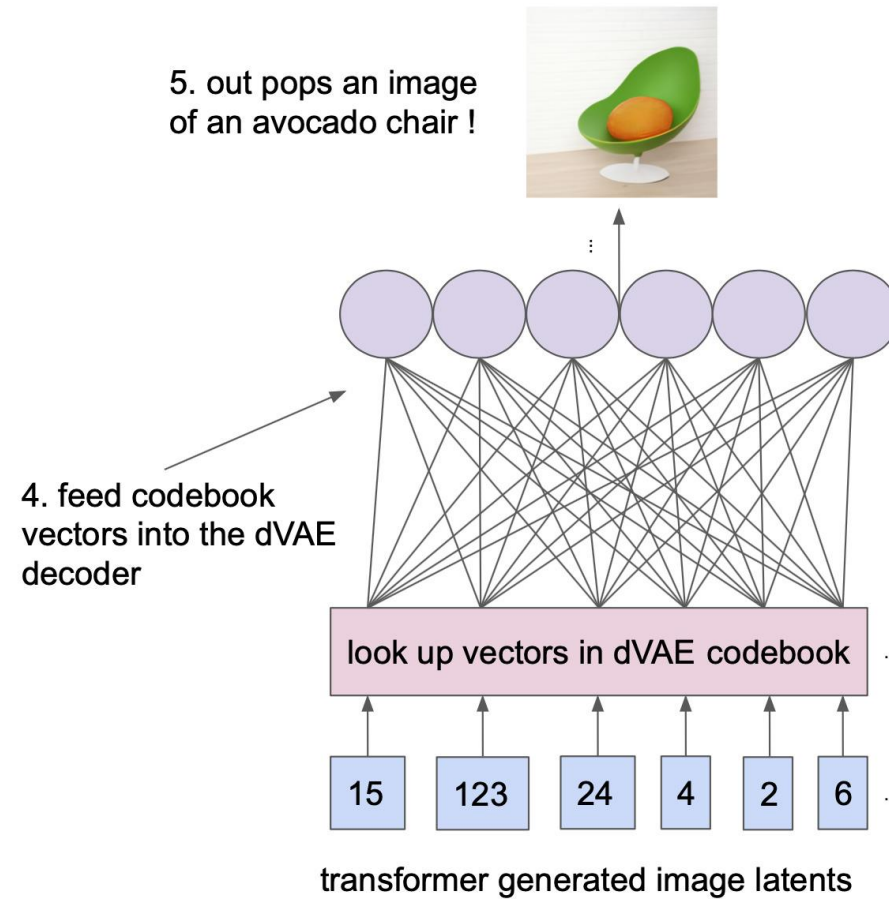


[View more or edit prompt ↓](#)

Learn to predict codebook vectors from text



Decode with the predicted vectors



Scaling high resolution

